

Derivative Calculator

Generated by Doxygen 1.17.0

1 Changelog	1
1.1 [Demo 1] - 2026-04-23	1
1.1.1 Added:	1
1.2 [Demo 1 v.2] - 2026-05-01	1
1.2.1 Added:	1
1.3 [Demo 2] - 2026-05-10	1
1.3.1 Added:	1
1.4 [Demo 3] - 2026-06-04	2
1.4.1 Added:	2
2 Derivative Calculator	3
2.1 Description	3
2.2 Installation	3
2.2.1 Prerequisites	3
2.3 Usage	3
2.4 Examples	4
2.5 Features	4
3 Hierarchical Index	5
3.1 Class Hierarchy	5
4 Class Index	7
4.1 Class List	7
5 File Index	9
5.1 File List	9
6 Class Documentation	11
6.1 AcoshNode Class Reference	11
6.1.1 Detailed Description	12
6.1.2 Member Function Documentation	12
6.1.2.1 Calculate()	12
6.1.2.2 Clone()	12
6.1.2.3 GetDerivative()	13
6.1.2.4 ToString()	13
6.1.2.5 UnaryNode()	13
6.2 AcosNode Class Reference	13
6.2.1 Detailed Description	14
6.2.2 Member Function Documentation	14
6.2.2.1 Calculate()	14
6.2.2.2 Clone()	15
6.2.2.3 GetDerivative()	15
6.2.2.4 ToString()	15
6.2.2.5 UnaryNode()	15
6.3 AddNode Class Reference	16

6.3.1 Detailed Description	16
6.3.2 Member Function Documentation	17
6.3.2.1 BinaryNode()	17
6.3.2.2 Calculate()	17
6.3.2.3 Clone()	17
6.3.2.4 GetDerivative()	17
6.3.2.5 ToString()	18
6.4 AsinhNode Class Reference	18
6.4.1 Detailed Description	19
6.4.2 Member Function Documentation	19
6.4.2.1 Calculate()	19
6.4.2.2 Clone()	19
6.4.2.3 GetDerivative()	20
6.4.2.4 ToString()	20
6.4.2.5 UnaryNode()	20
6.5 AsinNode Class Reference	20
6.5.1 Detailed Description	21
6.5.2 Member Function Documentation	21
6.5.2.1 Calculate()	21
6.5.2.2 Clone()	22
6.5.2.3 GetDerivative()	22
6.5.2.4 ToString()	22
6.5.2.5 UnaryNode()	22
6.6 AtanhNode Class Reference	23
6.6.1 Detailed Description	23
6.6.2 Member Function Documentation	24
6.6.2.1 Calculate()	24
6.6.2.2 Clone()	24
6.6.2.3 GetDerivative()	24
6.6.2.4 ToString()	25
6.6.2.5 UnaryNode()	25
6.7 AtanNode Class Reference	25
6.7.1 Detailed Description	26
6.7.2 Member Function Documentation	26
6.7.2.1 Calculate()	26
6.7.2.2 Clone()	26
6.7.2.3 GetDerivative()	27
6.7.2.4 ToString()	27
6.7.2.5 UnaryNode()	27
6.8 BinaryNode Class Reference	27
6.8.1 Detailed Description	28
6.8.2 Constructor & Destructor Documentation	28
6.8.2.1 BinaryNode()	28

6.8.3 Member Function Documentation	28
6.8.3.1 Calculate()	28
6.8.3.2 Clone()	29
6.8.3.3 GetDerivative()	29
6.8.3.4 ToString()	29
6.8.4 Member Data Documentation	30
6.8.4.1 left_child	30
6.8.4.2 right_child	30
6.9 CalculatorApp Class Reference	30
6.9.1 Detailed Description	32
6.9.2 Constructor & Destructor Documentation	32
6.9.2.1 CalculatorApp()	32
6.9.3 Member Function Documentation	32
6.9.3.1 AddSign()	32
6.9.3.2 FormulaInputCallback()	33
6.9.3.3 FunctionButton()	33
6.9.3.4 NumberButton()	33
6.9.3.5 ParseFormula()	34
6.9.3.6 Render()	34
6.9.3.7 RenderHelpMenu()	34
6.9.3.8 SetOfCalcButtons()	34
6.9.3.9 ShowFormulaPlot()	34
6.9.3.10 SignAndOperationButton()	35
6.9.3.11 TabOfFunctions()	35
6.9.4 Member Data Documentation	35
6.9.4.1 BUFFER_SIZE	35
6.9.4.2 derivative_formula	35
6.9.4.3 formula	35
6.9.4.4 formula_changed	36
6.9.4.5 head_derivative	36
6.9.4.6 head_formula	36
6.9.4.7 input_focus	36
6.9.4.8 last_cursor_position	36
6.9.4.9 parsed_formula	36
6.9.4.10 parsing_error	36
6.9.4.11 pos_reset	36
6.9.4.12 prev_limits	37
6.9.4.13 show_help_window	37
6.9.4.14 STEPS	37
6.9.4.15 window_flags	37
6.9.4.16 x_der_points	37
6.9.4.17 x_points	37
6.9.4.18 y_der_points	37

6.9.4.19 y_points	38
6.10 ConstNode Class Reference	38
6.10.1 Detailed Description	39
6.10.2 Constructor & Destructor Documentation	39
6.10.2.1 ConstNode() [1/2]	39
6.10.2.2 ConstNode() [2/2]	39
6.10.3 Member Function Documentation	39
6.10.3.1 Calculate()	39
6.10.3.2 Clone()	39
6.10.3.3 GetDerivative()	40
6.10.3.4 ToString()	40
6.10.4 Member Data Documentation	40
6.10.4.1 constant	40
6.11 CoshNode Class Reference	40
6.11.1 Detailed Description	41
6.11.2 Member Function Documentation	41
6.11.2.1 Calculate()	41
6.11.2.2 Clone()	42
6.11.2.3 GetDerivative()	42
6.11.2.4 ToString()	42
6.11.2.5 UnaryNode()	42
6.12 CosNode Class Reference	43
6.12.1 Detailed Description	43
6.12.2 Member Function Documentation	44
6.12.2.1 Calculate()	44
6.12.2.2 Clone()	44
6.12.2.3 GetDerivative()	44
6.12.2.4 ToString()	45
6.12.2.5 UnaryNode()	45
6.13 DivNode Class Reference	45
6.13.1 Detailed Description	46
6.13.2 Member Function Documentation	46
6.13.2.1 BinaryNode()	46
6.13.2.2 Calculate()	46
6.13.2.3 Clone()	47
6.13.2.4 GetDerivative()	47
6.13.2.5 ToString()	47
6.14 LnNode Class Reference	47
6.14.1 Detailed Description	48
6.14.2 Member Function Documentation	48
6.14.2.1 Calculate()	48
6.14.2.2 Clone()	49
6.14.2.3 GetDerivative()	49

6.14.2.4 ToString()	49
6.14.2.5 UnaryNode()	49
6.15 Log10Node Class Reference	50
6.15.1 Detailed Description	50
6.15.2 Member Function Documentation	51
6.15.2.1 Calculate()	51
6.15.2.2 Clone()	51
6.15.2.3 GetDerivative()	51
6.15.2.4 ToString()	52
6.15.2.5 UnaryNode()	52
6.16 LogNode Class Reference	52
6.16.1 Detailed Description	53
6.16.2 Member Function Documentation	53
6.16.2.1 BinaryNode()	53
6.16.2.2 Calculate()	53
6.16.2.3 Clone()	54
6.16.2.4 GetDerivative()	54
6.16.2.5 ToString()	54
6.17 MultNode Class Reference	55
6.17.1 Detailed Description	55
6.17.2 Member Function Documentation	56
6.17.2.1 BinaryNode()	56
6.17.2.2 Calculate()	56
6.17.2.3 Clone()	56
6.17.2.4 GetDerivative()	56
6.17.2.5 ToString()	57
6.18 Node Class Reference	57
6.18.1 Detailed Description	58
6.18.2 Constructor & Destructor Documentation	58
6.18.2.1 ~Node()	58
6.18.3 Member Function Documentation	58
6.18.3.1 Calculate()	58
6.18.3.2 Clone()	59
6.18.3.3 GetDerivative()	59
6.18.3.4 ToString()	59
6.19 PowNode Class Reference	60
6.19.1 Detailed Description	60
6.19.2 Member Function Documentation	61
6.19.2.1 BinaryNode()	61
6.19.2.2 Calculate()	61
6.19.2.3 Clone()	61
6.19.2.4 GetDerivative()	61
6.19.2.5 ToString()	62

6.20 SinhNode Class Reference	62
6.20.1 Detailed Description	63
6.20.2 Member Function Documentation	63
6.20.2.1 Calculate()	63
6.20.2.2 Clone()	63
6.20.2.3 GetDerivative()	64
6.20.2.4 ToString()	64
6.20.2.5 UnaryNode()	64
6.21 SinNode Class Reference	64
6.21.1 Detailed Description	65
6.21.2 Member Function Documentation	65
6.21.2.1 Calculate()	65
6.21.2.2 Clone()	66
6.21.2.3 GetDerivative()	66
6.21.2.4 ToString()	66
6.21.2.5 UnaryNode()	66
6.22 SqrtNode Class Reference	67
6.22.1 Detailed Description	67
6.22.2 Member Function Documentation	68
6.22.2.1 Calculate()	68
6.22.2.2 Clone()	68
6.22.2.3 GetDerivative()	68
6.22.2.4 ToString()	69
6.22.2.5 UnaryNode()	69
6.23 SubNode Class Reference	69
6.23.1 Detailed Description	70
6.23.2 Member Function Documentation	70
6.23.2.1 BinaryNode()	70
6.23.2.2 Calculate()	70
6.23.2.3 Clone()	71
6.23.2.4 GetDerivative()	71
6.23.2.5 ToString()	71
6.24 TanhNode Class Reference	72
6.24.1 Detailed Description	72
6.24.2 Member Function Documentation	73
6.24.2.1 Calculate()	73
6.24.2.2 Clone()	73
6.24.2.3 GetDerivative()	73
6.24.2.4 ToString()	74
6.24.2.5 UnaryNode()	74
6.25 TanNode Class Reference	74
6.25.1 Detailed Description	75
6.25.2 Member Function Documentation	75

6.25.2.1 Calculate()	75
6.25.2.2 Clone()	75
6.25.2.3 GetDerivative()	76
6.25.2.4 ToString()	76
6.25.2.5 UnaryNode()	76
6.26 Token Struct Reference	76
6.26.1 Detailed Description	77
6.26.2 Member Enumeration Documentation	77
6.26.2.1 TokenType	77
6.26.3 Constructor & Destructor Documentation	78
6.26.3.1 Token() [1/2]	78
6.26.3.2 Token() [2/2]	78
6.26.4 Member Function Documentation	78
6.26.4.1 operator=()	78
6.26.5 Member Data Documentation	79
6.26.5.1 token_type	79
6.26.5.2 val	79
6.27 UnaryNode Class Reference	79
6.27.1 Detailed Description	80
6.27.2 Constructor & Destructor Documentation	80
6.27.2.1 UnaryNode()	80
6.27.3 Member Function Documentation	80
6.27.3.1 Calculate()	80
6.27.3.2 Clone()	81
6.27.3.3 GetDerivative()	81
6.27.3.4 ToString()	82
6.27.4 Member Data Documentation	82
6.27.4.1 child	82
6.28 VariableNode Class Reference	82
6.28.1 Detailed Description	83
6.28.2 Member Function Documentation	83
6.28.2.1 Calculate()	83
6.28.2.2 Clone()	83
6.28.2.3 GetDerivative()	84
6.28.2.4 ToString()	84
7 File Documentation	85
7.1 CalculatorApp.cpp File Reference	85
7.2 CalculatorApp.h File Reference	85
7.3 CalculatorApp.h	85
7.4 CHANGELOG.md File Reference	86
7.5 FormulaTree.h File Reference	86
7.6 FormulaTree.h	88

7.7 main.cpp File Reference	92
7.7.1 Function Documentation	93
7.7.1.1 glfw_error_callback()	93
7.7.1.2 main()	93
7.8 Parser.cpp File Reference	93
7.8.1 Function Documentation	94
7.8.1.1 CheckToken()	94
7.8.1.2 CreateBinFunNode()	94
7.8.1.3 CreateFormulaTree()	95
7.8.1.4 CreateOperatorToken()	95
7.8.1.5 CreateUnFunNode()	95
7.8.1.6 getPriority()	96
7.8.1.7 is_numeric()	96
7.8.1.8 ModifyToRPN()	97
7.8.1.9 OperatorSign()	97
7.9 Parser.h File Reference	97
7.9.1 Function Documentation	98
7.9.1.1 CheckToken()	98
7.9.1.2 CreateBinFunNode()	99
7.9.1.3 CreateFormulaTree()	99
7.9.1.4 CreateOperatorToken()	99
7.9.1.5 CreateUnFunNode()	100
7.9.1.6 getPriority()	100
7.9.1.7 is_numeric()	101
7.9.1.8 ModifyToRPN()	101
7.9.1.9 OperatorSign()	101
7.10 Parser.h	102
7.11 README.md File Reference	102

Chapter 1

Changelog

1.1 [Demo 1] - 2026-04-23

1.1.1 Added:

- System window configuration.
- Graphical User Interface theme using Dear ImGui.
- Tab bar and equation input text field.
- Plotting view using ImPlot.

1.2 [Demo 1 v.2] - 2026-05-01

1.2.1 Added:

- Division into files.
- Improved User Interface.
- Improved inner workings.

1.3 [Demo 2] - 2026-05-10

1.3.1 Added:

- Implemented an engine to parse any given formula into a tree structure
- Creating a derivative tree based on the formula tree
- Visualisation of both formula and its derivative
- Engine supports basic math operations, trigonometric functions, hyperbolic functions, logarithms
- Added X and Y axis

1.4 [Demo 3] - 2026-06-04

1.4.1 Added:

- Handling wrong formulas
- Creating a string for derivative formula from derivative tree
- Added a "Help" window with instructions how to use the application
- Brought back special signs: $,$ $\times$ and $\div$

Chapter 2

Derivative Calculator

2.1 Description

The Derivative Calculator is a desktop application developed in C++ that allows users to parse mathematical equations, calculate their derivatives, and visualize both functions simultaneously in real-time. The project utilizes the Dear ImGui framework for the graphical user interface and the ImPlot library for rendering mathematical graphs. Under the hood, it implements a custom mathematical engine based on Reverse Polish Notation (RPN) and an Abstract Syntax Tree (AST) to process inputs, apply symbolic differentiation rules, and algebraically simplify the resulting expressions.

2.2 Installation

To compile and run this project, you need a standard C++ development environment.

2.2.1 Prerequisites

- C++17 Standard or higher.
- IDE: Visual Studio 2022 (recommended for Windows users) or any compatible C++ compiler.
- Dependencies: The project requires the Dear ImGui and ImPlot libraries. (Note: Ensure that the include directories and linkers for these libraries are properly configured in your project properties if they are not already bundled).

2.3 Usage

This application is highly useful for students, educators, and engineers who need to quickly visualize mathematical functions and analyze their rate of change (derivatives).

To use the calculator, simply interact with the on-screen keyboard or use your physical keyboard to type a valid mathematical function in terms of the independent variable x . The application will instantly parse the formula, generate the derivative, and plot both lines. You can freely pan the camera by clicking and dragging the graph, or zoom in and out using the mouse wheel. An integrated "Help" menu is available within the application.

2.4 Examples

A standard use case involves analyzing complex trigonometric and polynomial bounds. For instance, inputting the formula $\sin(\text{pow}(x, 2)) + 2 * x$ will automatically compute the exact derivative $\cos(\text{pow}(x, 2)) * 2 * x + 2$ (simplified through the AST engine) and display their respective intersections on the graph.

2.5 Features

- Custom AST Parsing Engine: Converts standard infix mathematical notation into Reverse Polish Notation and builds a traversable syntax tree.
- Symbolic Differentiation: Dynamically applies the chain rule, product rule, and quotient rule to calculate exact derivatives instead of relying on numerical approximations.
- Algebraic Simplification: Automatically reduces mathematical clutter (e.g., multiplying by zero or one) for cleaner derivative output.
- Real-Time Graphing: Rendering of multiple functions using ImPlot.
- Comprehensive Math Support: Supports basic arithmetic, trigonometry, cyclometric functions, hyperbolic functions, logarithms, powers and natural constants (, e).
- Robust Error Handling: Instantly detects parsing errors, missing brackets, or invalid syntax and alerts the user through the UI without crashing the application.

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

CalculatorApp	30
Node	57
BinaryNode	27
AddNode	16
DivNode	45
LogNode	52
MultNode	55
PowNode	60
SubNode	69
ConstNode	38
UnaryNode	79
AcosNode	13
AcoshNode	11
AsinNode	20
AsinhNode	18
AtanNode	25
AtanhNode	23
CosNode	43
CoshNode	40
LnNode	47
Log10Node	50
SinNode	64
SinhNode	62
SqrtNode	67
TanNode	74
TanhNode	72
VariableNode	82
Token	76

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AcoshNode	
Node representing an inverse hyperbolic cosine operation	11
AcosNode	
Node representing an arccosine operation	13
AddNode	
Node representing an addition operation	16
AsinhNode	
Node representing an inverse hyperbolic sine operation	18
AsinNode	
Node representing an arcsine operation	20
AtanhNode	
Node representing an inverse hyperbolic tangent operation	23
AtanNode	
Node representing an arctangent operation	25
BinaryNode	
Abstract base class representing binary mathematical operations	27
CalculatorApp	
Class responsible for rendering the elements of an app and drawing a plot	30
ConstNode	
Node of the formula tree for constant value	38
CoshNode	
Node representing a hyperbolic cosine operation	40
CosNode	
Node representing a cosine operation	43
DivNode	
Node representing a division operation	45
LnNode	
Node representing a dnatural logarithm operation	47
Log10Node	
Node representing a logarithm with a base 10 operation	50
LogNode	
Node representing a logarithm operation with any base	52
MultNode	
Node representing a multiplication operation	55
Node	
Base node class for the mathematical formula tree	57

PowNode	
Node representing a exponentiation operation	60
SinhNode	
Node representing a hyperbolic sine operation	62
SinNode	
Node representing a sine operation	64
SqrtNode	
Node representing a square root operation	67
SubNode	
Node representing a substraction operation	69
TanhNode	
Node representing a hyperbolic tangent operation	72
TanNode	
Node representing a tangent operation	74
Token	
Represents a single token generated during the parsing of a formula	76
UnaryNode	
Abstract base class representing unary mathematical operations	79
VariableNode	
Node of the formula tree for variable x	82

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

CalculatorApp.cpp	85
CalculatorApp.h	85
FormulaTree.h	86
main.cpp	92
Parser.cpp	93
Parser.h	97

Chapter 6

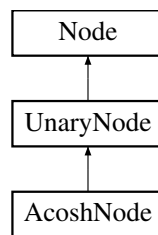
Class Documentation

6.1 AcoshNode Class Reference

[Node](#) representing an inverse hyperbolic cosine operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for AcoshNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [UnaryNode](#)

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- `std::unique_ptr< Node > child`

6.1.1 Detailed Description

[Node](#) representing an inverse hyperbolic cosine operation.

6.1.2 Member Function Documentation

6.1.2.1 Calculate()

```
float AcoshNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.1.2.2 Clone()

```
std::unique_ptr< Node > AcoshNode::Clone () const    [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- `std::unique_ptr<Node>` A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.1.2.3 GetDerivative()

```
std::unique_ptr< Node > AcoshNode::GetDerivative () const [inline], [override], [virtual]
```

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- `std::unique_ptr<Node>` A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.1.2.4 ToString()

```
std::string AcoshNode::ToString () const [inline], [override], [virtual]
```

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [UnaryNode](#).

6.1.2.5 UnaryNode()

```
UnaryNode::UnaryNode (  
    std::unique_ptr< Node > node = nullptr) [inline]
```

The documentation for this class was generated from the following file:

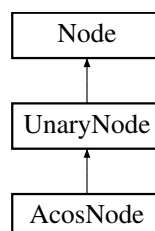
- [FormulaTree.h](#)

6.2 AcosNode Class Reference

[Node](#) representing an arccosine operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for AcosNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [UnaryNode](#)

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- std::unique_ptr< [Node](#) > [child](#)

6.2.1 Detailed Description

[Node](#) representing an arccosine operation.

6.2.2 Member Function Documentation

6.2.2.1 Calculate()

```
float AcosNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.2.2.2 Clone()

`std::unique_ptr< Node > AcosNode::Clone () const` `[inline]`, `[override]`, `[virtual]`

Creates a deep copy of the node.

Returns

- `std::unique_ptr<Node>` A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.2.2.3 GetDerivative()

`std::unique_ptr< Node > AcosNode::GetDerivative () const` `[inline]`, `[override]`, `[virtual]`

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- `std::unique_ptr<Node>` A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.2.2.4 ToString()

`std::string AcosNode::ToString () const` `[inline]`, `[override]`, `[virtual]`

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [UnaryNode](#).

6.2.2.5 UnaryNode()

[UnaryNode::UnaryNode](#) (
 `std::unique_ptr< Node > node = nullptr`) `[inline]`

The documentation for this class was generated from the following file:

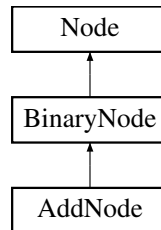
- [FormulaTree.h](#)

6.3 AddNode Class Reference

[Node](#) representing an addition operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for AddNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [BinaryNode](#) (std::unique_ptr< [Node](#) > left_node=nullptr, std::unique_ptr< [Node](#) > right_node=nullptr)

Public Member Functions inherited from [BinaryNode](#)

- [BinaryNode](#) (std::unique_ptr< [Node](#) > left_node=nullptr, std::unique_ptr< [Node](#) > right_node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [BinaryNode](#)

- std::unique_ptr< [Node](#) > [left_child](#)
- std::unique_ptr< [Node](#) > [right_child](#)

6.3.1 Detailed Description

[Node](#) representing an addition operation.

6.3.2 Member Function Documentation

6.3.2.1 BinaryNode()

```
BinaryNode::BinaryNode (
    std::unique_ptr< Node > left_node = nullptr,
    std::unique_ptr< Node > right_node = nullptr) [inline]
```

6.3.2.2 Calculate()

```
float AddNode::Calculate (
    float x) const [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [BinaryNode](#).

6.3.2.3 Clone()

```
std::unique_ptr< Node > AddNode::Clone () const [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [BinaryNode](#).

6.3.2.4 GetDerivative()

```
std::unique_ptr< Node > AddNode::GetDerivative () const [inline], [override], [virtual]
```

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- std::unique_ptr<Node> A smart pointer to the root of the derivative tree.

Implements [BinaryNode](#).

6.3.2.5 ToString()

`std::string AddNode::ToString () const [inline], [override], [virtual]`

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [BinaryNode](#).

The documentation for this class was generated from the following file:

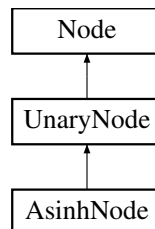
- [FormulaTree.h](#)

6.4 AsinhNode Class Reference

[Node](#) representing an inverse hyperbolic sine operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for AsinhNode:



Public Member Functions

- `float Calculate (float x) const override`
Calculates the value of the function at a given point.
- `std::unique_ptr< Node > Clone () const override`
Creates a deep copy of the node.
- `std::unique_ptr< Node > GetDerivative () const override`
Generates the derivative tree for this node.
- `std::string ToString () const override`
Converts the tree structure into a string.
- `UnaryNode (std::unique_ptr< Node > node=nullptr)`

Public Member Functions inherited from [UnaryNode](#)

- `UnaryNode (std::unique_ptr< Node > node=nullptr)`

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- std::unique_ptr< [Node](#) > child

6.4.1 Detailed Description

[Node](#) representing an inverse hyperbolic sine operation.

6.4.2 Member Function Documentation

6.4.2.1 Calculate()

```
float AsinhNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.4.2.2 Clone()

```
std::unique_ptr< Node > AsinhNode::Clone () const    [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.4.2.3 GetDerivative()

```
std::unique_ptr< Node > AsinhNode::GetDerivative () const [inline], [override], [virtual]
```

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- `std::unique_ptr<Node>` A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.4.2.4 ToString()

```
std::string AsinhNode::ToString () const [inline], [override], [virtual]
```

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [UnaryNode](#).

6.4.2.5 UnaryNode()

```
UnaryNode::UnaryNode (  
    std::unique_ptr< Node > node = nullptr) [inline]
```

The documentation for this class was generated from the following file:

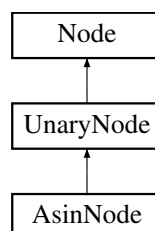
- [FormulaTree.h](#)

6.5 AsinNode Class Reference

[Node](#) representing an arcsine operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for AsinNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [UnaryNode](#)

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- std::unique_ptr< [Node](#) > [child](#)

6.5.1 Detailed Description

[Node](#) representing an arcsine operation.

6.5.2 Member Function Documentation

6.5.2.1 Calculate()

```
float AsinNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.5.2.2 Clone()

`std::unique_ptr< Node > AsinNode::Clone () const` `[inline]`, `[override]`, `[virtual]`

Creates a deep copy of the node.

Returns

- `std::unique_ptr<Node>` A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.5.2.3 GetDerivative()

`std::unique_ptr< Node > AsinNode::GetDerivative () const` `[inline]`, `[override]`, `[virtual]`

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- `std::unique_ptr<Node>` A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.5.2.4 ToString()

`std::string AsinNode::ToString () const` `[inline]`, `[override]`, `[virtual]`

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [UnaryNode](#).

6.5.2.5 UnaryNode()

[UnaryNode::UnaryNode](#) (
 `std::unique_ptr< Node > node = nullptr`) `[inline]`

The documentation for this class was generated from the following file:

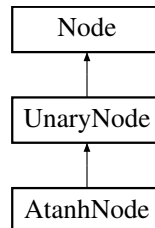
- [FormulaTree.h](#)

6.6 AtanhNode Class Reference

[Node](#) representing an inverse hyperbolic tangent operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for AtanhNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [UnaryNode](#)

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- std::unique_ptr< [Node](#) > [child](#)

6.6.1 Detailed Description

[Node](#) representing an inverse hyperbolic tangent operation.

6.6.2 Member Function Documentation

6.6.2.1 Calculate()

```
float AtanhNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.6.2.2 Clone()

```
std::unique_ptr< Node > AtanhNode::Clone () const    [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.6.2.3 GetDerivative()

```
std::unique_ptr< Node > AtanhNode::GetDerivative () const    [inline], [override], [virtual]
```

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- std::unique_ptr<Node> A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.6.2.4 ToString()

`std::string AtanhNode::ToString () const` [inline], [override], [virtual]

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [UnaryNode](#).

6.6.2.5 UnaryNode()

`UnaryNode::UnaryNode (`
`std::unique_ptr< Node > node = nullptr)` [inline]

The documentation for this class was generated from the following file:

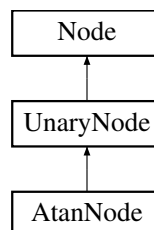
- [FormulaTree.h](#)

6.7 AtanNode Class Reference

[Node](#) representing an arctangent operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for AtanNode:



Public Member Functions

- `float Calculate (float x) const` override
 Calculates the value of the function at a given point.
- `std::unique_ptr< Node > Clone ()` const override
 Creates a deep copy of the node.
- `std::unique_ptr< Node > GetDerivative ()` const override
 Generates the derivative tree for this node.
- `std::string ToString ()` const override
 Converts the tree structure into a string.
- `UnaryNode (std::unique_ptr< Node > node=nullptr)`

Public Member Functions inherited from [UnaryNode](#)

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- std::unique_ptr< [Node](#) > [child](#)

6.7.1 Detailed Description

[Node](#) representing an arctangent operation.

6.7.2 Member Function Documentation

6.7.2.1 Calculate()

```
float AtanNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.7.2.2 Clone()

```
std::unique_ptr< Node > AtanNode::Clone () const    [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.7.2.3 GetDerivative()

```
std::unique_ptr< Node > AtanNode::GetDerivative () const [inline], [override], [virtual]
```

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- `std::unique_ptr<Node>` A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.7.2.4 ToString()

```
std::string AtanNode::ToString () const [inline], [override], [virtual]
```

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [UnaryNode](#).

6.7.2.5 UnaryNode()

```
UnaryNode::UnaryNode (
    std::unique_ptr< Node > node = nullptr) [inline]
```

The documentation for this class was generated from the following file:

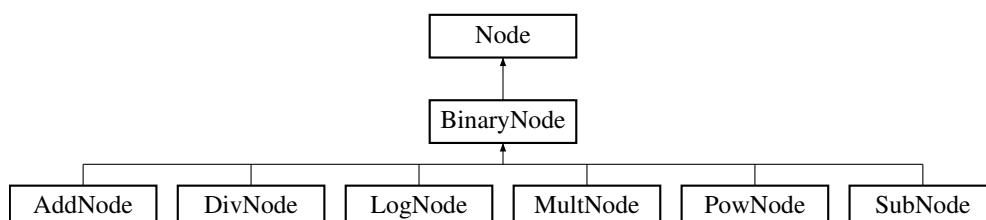
- [FormulaTree.h](#)

6.8 BinaryNode Class Reference

Abstract base class representing binary mathematical operations.

```
#include <FormulaTree.h>
```

Inheritance diagram for BinaryNode:



Public Member Functions

- [BinaryNode](#) (std::unique_ptr< [Node](#) > left_node=nullptr, std::unique_ptr< [Node](#) > right_node=nullptr)
- virtual float [Calculate](#) (float x) const =0
Calculates the value of the function at a given point.
- virtual std::unique_ptr< [Node](#) > [Clone](#) () const =0
Creates a deep copy of the node.
- virtual std::unique_ptr< [Node](#) > [GetDerivative](#) () const =0
Generates the derivative tree for this node.
- virtual std::string [ToString](#) () const =0
Converts the tree structure into a string.

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Protected Attributes

- std::unique_ptr< [Node](#) > [left_child](#)
- std::unique_ptr< [Node](#) > [right_child](#)

6.8.1 Detailed Description

Abstract base class representing binary mathematical operations.

6.8.2 Constructor & Destructor Documentation

6.8.2.1 BinaryNode()

```
BinaryNode::BinaryNode (
    std::unique_ptr< Node > left_node = nullptr,
    std::unique_ptr< Node > right_node = nullptr) [inline]
```

6.8.3 Member Function Documentation

6.8.3.1 Calculate()

```
virtual float BinaryNode::Calculate (
    float x) const [pure virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [Node](#).

Implemented in [AddNode](#), [DivNode](#), [LogNode](#), [MultNode](#), [PowNode](#), and [SubNode](#).

6.8.3.2 Clone()

```
virtual std::unique_ptr< Node > BinaryNode::Clone () const [pure virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [Node](#).

Implemented in [AddNode](#), [DivNode](#), [LogNode](#), [MultNode](#), [PowNode](#), and [SubNode](#).

6.8.3.3 GetDerivative()

```
virtual std::unique_ptr< Node > BinaryNode::GetDerivative () const [pure virtual]
```

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- std::unique_ptr<Node> A smart pointer to the root of the derivative tree.

Implements [Node](#).

Implemented in [AddNode](#), [DivNode](#), [LogNode](#), [MultNode](#), [PowNode](#), and [SubNode](#).

6.8.3.4 ToString()

```
virtual std::string BinaryNode::ToString () const [pure virtual]
```

Converts the tree structure into a string.

Returns

std::string The string representation of the formula.

Implements [Node](#).

Implemented in [AddNode](#), [DivNode](#), [LogNode](#), [MultNode](#), [PowNode](#), and [SubNode](#).

6.8.4 Member Data Documentation

6.8.4.1 left_child

`std::unique_ptr<Node> BinaryNode::left_child` [protected]

6.8.4.2 right_child

`std::unique_ptr<Node> BinaryNode::right_child` [protected]

The documentation for this class was generated from the following file:

- [FormulaTree.h](#)

6.9 CalculatorApp Class Reference

Class responsible for rendering the elements of an app and drawing a plot.

```
#include <CalculatorApp.h>
```

Public Member Functions

- [CalculatorApp](#) ()
Default constructor for the [CalculatorApp](#).
- void [Render](#) ()
Renders all of the calculator app and its components.

Private Member Functions

- void [AddSign](#) (const char *sign)
Inserts a number, sign or function into formula string.
- bool [NumberButton](#) (const char *sign)
Renders one button that has a number on it.
- bool [SignAndOperationButton](#) (const char *sign)
Renders one button that has a constant or an operation sign on it.
- bool [FunctionButton](#) (const char *sign)
Renders one button that has a function on it.
- void [SetOfCalcButtons](#) ()
Renders all of the NumberButtons and SignAndOperationButton.
- void [TabOfFunctions](#) (float width)
Renders a table of all of the FunctionButtons.
- void [RenderHelpMenu](#) ()
Render the "Help" window.
- void [ParseFormula](#) ()
Divides the inputted formula into tokens.
- void [ShowFormulaPlot](#) ()
Renders the graph of a function.

Static Private Member Functions

- static int [FormulaInputCallback](#) (ImGuiInputTextCallbackData *data)
Callback function for handling the formula text input field.

Private Attributes

- bool [input_focus](#) = true
Flag used to force or maintain focus on the text input field.
- bool [formula_changed](#) = false
Flag indicating that formula has changed.
- int [last_cursor_position](#) = 0
Stores the last position of the cursor in the input field.
- bool [show_help_window](#) = false
Flag toggling the visibility of the help window.
- bool [parsing_error](#) = false
Displays an error message if true.
- bool [pos_reset](#) = false
Flag indicating if the plot camera position should be reset.
- std::vector< float > [x_points](#)
X-axis coordinates for the main plot.
- std::vector< float > [y_points](#)
Y-axis coordinates for the main plot.
- std::vector< float > [x_der_points](#)
X-axis coordinates for the derivative plot.
- std::vector< float > [y_der_points](#)
Y-axis coordinates for the derivative plot.
- std::string [formula](#) = ""
User's input.
- std::string [parsed_formula](#) = ""
Formula inputted by the user.
- std::string [derivative_formula](#) = ""
Formula of the derivative.
- std::unique_ptr< [Node](#) > [head_formula](#) = nullptr
Smart pointer to the root node of the primary formula tree.
- std::unique_ptr< [Node](#) > [head_derivative](#) = nullptr
Smart pointer to the root node of the derivative tree.
- ImGuiWindowFlags [window_flags](#)
Configuration flags for the main ImGui app.
- ImPlotRect [prev_limits](#)
Stores the previous plot bounding box to detect camera movement.

Static Private Attributes

- static constexpr int [STEPS](#) = 10001
Number of points that are drawn on the graph.
- static constexpr int [BUFFER_SIZE](#) = 64
The maximum size of formula that can be imputed.

6.9.1 Detailed Description

Class responsible for rendering the elements of an app and drawing a plot.

- This class stores variables needed for the rendering of an app, managing a text input, displaying formulas, calculating points on the graph as well as ImGui and ImPlot flags. It has methods for rendering, managing a user interface and drawing a graph.

6.9.2 Constructor & Destructor Documentation

6.9.2.1 CalculatorApp()

CalculatorApp::CalculatorApp ()

Default constructor for the [CalculatorApp](#).

- Initializes the default application state, configures ImGui window flags, and prepares the mathematical data buffers for rendering.

6.9.3 Member Function Documentation

6.9.3.1 AddSign()

```
void CalculatorApp::AddSign (
    const char * sign) [private]
```

Inserts a number, sign or function into formula string.

- This function inserts a text from a button that was pressed by the user into the formula string. Before that it checks if there is enough free space for that text to be put into the string.

- Parameters

sign	Text from the button.
------	-----------------------

6.9.3.2 FormulaInputCallback()

```
int CalculatorApp::FormulaInputCallback (
    ImGuiInputTextCallbackData * data) [static], [private]
```

Callback function for handling the formula text input field.

- This method is triggered by ImGui during text input operations. It is primarily used to manage the text cursor state and maintain input focus.

- Parameters

data	Pointer to the ImGui callback data structure containing the current state of the text input.
------	--

Returns

int Returns 0 to indicate successful execution.

6.9.3.3 FunctionButton()

```
bool CalculatorApp::FunctionButton (
    const char * sign) [private]
```

Renders one button that has a function on it.

- Function displays the sign parameter on the button. It changes color when is hovered and return true value if it's pressed.

- Parameters

sign	The text displayed on the button.
------	-----------------------------------

Returns

bool Indicates if the button was pressed.

6.9.3.4 NumberButton()

```
bool CalculatorApp::NumberButton (
    const char * sign) [private]
```

Renders one button that has a number on it.

- Function displays the sign parameter on the button. It changes color when is hovered and return true value if it's pressed.

- Parameters

sign	The text displayed on the button.
------	-----------------------------------

Returns

bool Indicates if the button was pressed.

6.9.3.5 ParseFormula()

```
void CalculatorApp::ParseFormula () [private]
```

Divides the inputted formula into tokens.

- This function divides the formula inserted by the user and creates a vector of tokens that represent singular operations, constants or variable. Afterwards it sends it to create a formula tree from it.

6.9.3.6 Render()

```
void CalculatorApp::Render (
    void )
```

Renders all of the calculator app and its components.

6.9.3.7 RenderHelpMenu()

```
void CalculatorApp::RenderHelpMenu () [private]
```

Render the "Help" window.

- Displays a "Help" window with tips and guidance for the user.

6.9.3.8 SetOfCalcButtons()

```
void CalculatorApp::SetOfCalcButtons () [private]
```

Renders all of the NumberButtons and SignAndOperationButton.

6.9.3.9 ShowFormulaPlot()

```
void CalculatorApp::ShowFormulaPlot () [private]
```

Renders the graph of a function.

- It checks coordinates that are seen by the user and calculates the points of the formula as well as the derivative. Only starts to calculate the points again if the camera was moved. It also has a mechanism to avoid creating asymptotes and interruptions in the plot.

6.9.3.10 SignAndOperationButton()

```
bool CalculatorApp::SignAndOperationButton (
    const char * sign)    [private]
```

Renders one button that has a constant or an operation sign on it.

- Function displays the sign parameter on the button. It changes color when is hovered and return true value if it's pressed.

- Parameters

sign	The text displayed on the button.
------	-----------------------------------

Returns

bool Indicates if the button was pressed.

6.9.3.11 TabOfFunctions()

```
void CalculatorApp::TabOfFunctions (
    float width)    [private]
```

Renders a table of all of the FunctionButtons.

- Parameters

width	The size of the table.
-------	------------------------

6.9.4 Member Data Documentation

6.9.4.1 BUFFER_SIZE

```
int CalculatorApp::BUFFER_SIZE = 64    [static], [constexpr], [private]
```

The maximum size of formula that can be imputed.

6.9.4.2 derivative_formula

```
std::string CalculatorApp::derivative_formula = ""    [private]
```

Formula of the derivative.

6.9.4.3 formula

```
std::string CalculatorApp::formula = ""    [private]
```

User's input.

6.9.4.4 formula_changed

```
bool CalculatorApp::formula_changed = false [private]
```

Flag indicating that formula has changed.

6.9.4.5 head_derivative

```
std::unique_ptr<Node> CalculatorApp::head_derivative = nullptr [private]
```

Smart pointer to the root node of the derivative tree.

6.9.4.6 head_formula

```
std::unique_ptr<Node> CalculatorApp::head_formula = nullptr [private]
```

Smart pointer to the root node of the primary formula tree.

6.9.4.7 input_focus

```
bool CalculatorApp::input_focus = true [private]
```

Flag used to force or maintain focus on the text input field.

6.9.4.8 last_cursor_position

```
int CalculatorApp::last_cursor_position = 0 [private]
```

Stores the last position of the cursor in the input field.

6.9.4.9 parsed_formula

```
std::string CalculatorApp::parsed_formula = "" [private]
```

Formula inputted by the user.

6.9.4.10 parsing_error

```
bool CalculatorApp::parsing_error = false [private]
```

Displays an error message if true.

6.9.4.11 pos_reset

```
bool CalculatorApp::pos_reset = false [private]
```

Flag indicating if the plot camera position should be reset.

6.9.4.12 prev_limits

ImPlotRect CalculatorApp::prev_limits [private]

Stores the previous plot bounding box to detect camera movement.

6.9.4.13 show_help_window

bool CalculatorApp::show_help_window = false [private]

Flag toggling the visibility of the help window.

6.9.4.14 STEPS

int CalculatorApp::STEPS = 10001 [static], [constexpr], [private]

Number of points that are drawn on the graph.

6.9.4.15 window_flags

ImGuiWindowFlags CalculatorApp::window_flags [private]

Configuration flags for the main ImGui app.

6.9.4.16 x_der_points

std::vector<float> CalculatorApp::x_der_points [private]

X-axis coordinates for the derivative plot.

6.9.4.17 x_points

std::vector<float> CalculatorApp::x_points [private]

X-axis coordinates for the main plot.

6.9.4.18 y_der_points

std::vector<float> CalculatorApp::y_der_points [private]

Y-axis coordinates for the derivative plot.

6.9.4.19 y_points

```
std::vector<float> CalculatorApp::y_points [private]
```

Y-axis coordinates for the main plot.

The documentation for this class was generated from the following files:

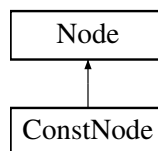
- [CalculatorApp.h](#)
- [CalculatorApp.cpp](#)

6.10 ConstNode Class Reference

[Node](#) of the formula tree for constant value.

```
#include <FormulaTree.h>
```

Inheritance diagram for ConstNode:



Public Member Functions

- [ConstNode](#) (float val)
- [ConstNode](#) (std::string str)
- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Private Attributes

- float [constant](#)
The value of the constant node.

6.10.1 Detailed Description

[Node](#) of the formula tree for constant value.

6.10.2 Constructor & Destructor Documentation

6.10.2.1 ConstNode() [1/2]

```
ConstNode::ConstNode (
    float val) [inline]
```

6.10.2.2 ConstNode() [2/2]

```
ConstNode::ConstNode (
    std::string str) [inline]
```

6.10.3 Member Function Documentation

6.10.3.1 Calculate()

```
float ConstNode::Calculate (
    float x) const [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [Node](#).

6.10.3.2 Clone()

```
std::unique_ptr<Node> ConstNode::Clone () const [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [Node](#).

6.10.3.3 GetDerivative()

```
std::unique_ptr< Node > ConstNode::GetDerivative () const [inline], [override], [virtual]
```

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- `std::unique_ptr<Node>` A smart pointer to the root of the derivative tree.

Implements [Node](#).

6.10.3.4 ToString()

```
std::string ConstNode::ToString () const [inline], [override], [virtual]
```

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [Node](#).

6.10.4 Member Data Documentation

6.10.4.1 constant

```
float ConstNode::constant [private]
```

The value of the constant node.

The documentation for this class was generated from the following file:

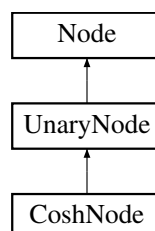
- [FormulaTree.h](#)

6.11 CoshNode Class Reference

[Node](#) representing a hyperbolic cosine operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for CoshNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [UnaryNode](#)

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- std::unique_ptr< [Node](#) > [child](#)

6.11.1 Detailed Description

[Node](#) representing a hyperbolic cosine operation.

6.11.2 Member Function Documentation

6.11.2.1 Calculate()

```
float CoshNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.11.2.2 Clone()

`std::unique_ptr< Node > CoshNode::Clone () const` `[inline]`, `[override]`, `[virtual]`

Creates a deep copy of the node.

Returns

- `std::unique_ptr<Node>` A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.11.2.3 GetDerivative()

`std::unique_ptr< Node > CoshNode::GetDerivative () const` `[inline]`, `[override]`, `[virtual]`

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- `std::unique_ptr<Node>` A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.11.2.4 ToString()

`std::string CoshNode::ToString () const` `[inline]`, `[override]`, `[virtual]`

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [UnaryNode](#).

6.11.2.5 UnaryNode()

[UnaryNode::UnaryNode](#) (
 `std::unique_ptr< Node > node = nullptr`) `[inline]`

The documentation for this class was generated from the following file:

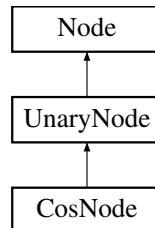
- [FormulaTree.h](#)

6.12 CosNode Class Reference

[Node](#) representing a cosine operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for CosNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [UnaryNode](#)

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- std::unique_ptr< [Node](#) > [child](#)

6.12.1 Detailed Description

[Node](#) representing a cosine operation.

6.12.2 Member Function Documentation

6.12.2.1 Calculate()

```
float CosNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.12.2.2 Clone()

```
std::unique_ptr< Node > CosNode::Clone () const    [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.12.2.3 GetDerivative()

```
std::unique_ptr< Node > CosNode::GetDerivative () const    [inline], [override], [virtual]
```

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- std::unique_ptr<Node> A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.12.2.4 ToString()

```
std::string CosNode::ToString () const [inline], [override], [virtual]
```

Converts the tree structure into a string.

Returns

std::string The string representation of the formula.

Implements [UnaryNode](#).

6.12.2.5 UnaryNode()

```
UnaryNode::UnaryNode (
    std::unique_ptr< Node > node = nullptr) [inline]
```

The documentation for this class was generated from the following file:

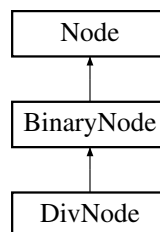
- [FormulaTree.h](#)

6.13 DivNode Class Reference

[Node](#) representing a division operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for DivNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the division of the left child by the right child.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node. Uses the formula $(f / g)' = (f' * g - f * g') / g^2$.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [BinaryNode](#) (std::unique_ptr< [Node](#) > left_node=nullptr, std::unique_ptr< [Node](#) > right_node=nullptr)

Public Member Functions inherited from [BinaryNode](#)

- [BinaryNode](#) (std::unique_ptr< [Node](#) > left_node=nullptr, std::unique_ptr< [Node](#) > right_node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [BinaryNode](#)

- std::unique_ptr< [Node](#) > left_child
- std::unique_ptr< [Node](#) > right_child

6.13.1 Detailed Description

[Node](#) representing a division operation.

6.13.2 Member Function Documentation

6.13.2.1 [BinaryNode](#)()

```
BinaryNode::BinaryNode (
    std::unique_ptr< Node > left_node = nullptr,
    std::unique_ptr< Node > right_node = nullptr) [inline]
```

6.13.2.2 [Calculate](#)()

```
float DivNode::Calculate (
    float x) const [inline], [override], [virtual]
```

Calculates the division of the left child by the right child.

- Protects against division by zero by returning NAN.
- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The result of division, or NAN if the denominator is 0.

Implements [BinaryNode](#).

6.13.2.3 Clone()

`std::unique_ptr< Node > DivNode::Clone () const` [inline], [override], [virtual]

Creates a deep copy of the node.

Returns

- `std::unique_ptr<Node>` A smart pointer to the cloned node.

Implements [BinaryNode](#).

6.13.2.4 GetDerivative()

`std::unique_ptr< Node > DivNode::GetDerivative () const` [inline], [override], [virtual]

Generates the derivative tree for this node. Uses the formula $(f / g)' = (f' * g - f * g') / g^2$.

Returns

- `std::unique_ptr<Node>` A smart pointer to a division node.

Implements [BinaryNode](#).

6.13.2.5 ToString()

`std::string DivNode::ToString () const` [inline], [override], [virtual]

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [BinaryNode](#).

The documentation for this class was generated from the following file:

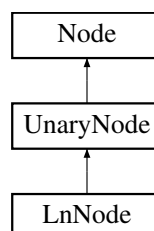
- [FormulaTree.h](#)

6.14 LnNode Class Reference

[Node](#) representing a natural logarithm operation.

`#include <FormulaTree.h>`

Inheritance diagram for LnNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [UnaryNode](#)

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- std::unique_ptr< [Node](#) > [child](#)

6.14.1 Detailed Description

[Node](#) representing a dnatural logarithm operation.

6.14.2 Member Function Documentation

6.14.2.1 Calculate()

```
float LnNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.14.2.2 Clone()

`std::unique_ptr< Node > LnNode::Clone () const` `[inline]`, `[override]`, `[virtual]`

Creates a deep copy of the node.

Returns

- `std::unique_ptr<Node>` A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.14.2.3 GetDerivative()

`std::unique_ptr< Node > LnNode::GetDerivative () const` `[inline]`, `[override]`, `[virtual]`

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- `std::unique_ptr<Node>` A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.14.2.4 ToString()

`std::string LnNode::ToString () const` `[inline]`, `[override]`, `[virtual]`

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [UnaryNode](#).

6.14.2.5 UnaryNode()

[UnaryNode::UnaryNode](#) (
 `std::unique_ptr< Node > node = nullptr`) `[inline]`

The documentation for this class was generated from the following file:

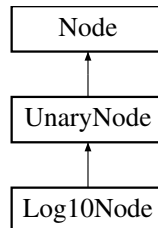
- [FormulaTree.h](#)

6.15 Log10Node Class Reference

[Node](#) representing a logarithm with a base 10 operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for Log10Node:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [UnaryNode](#)

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- std::unique_ptr< [Node](#) > [child](#)

6.15.1 Detailed Description

[Node](#) representing a logarithm with a base 10 operation.

6.15.2 Member Function Documentation

6.15.2.1 Calculate()

```
float Log10Node::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.15.2.2 Clone()

```
std::unique_ptr< Node > Log10Node::Clone () const    [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.15.2.3 GetDerivative()

```
std::unique_ptr< Node > Log10Node::GetDerivative () const    [inline], [override], [virtual]
```

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- std::unique_ptr<Node> A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.15.2.4 ToString()

`std::string Log10Node::ToString () const` [inline], [override], [virtual]

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [UnaryNode](#).

6.15.2.5 UnaryNode()

`UnaryNode::UnaryNode (`
`std::unique_ptr< Node > node = nullptr)` [inline]

The documentation for this class was generated from the following file:

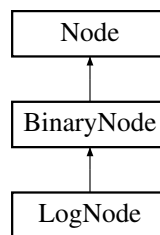
- [FormulaTree.h](#)

6.16 LogNode Class Reference

[Node](#) representing a logarithm operation with any base.

```
#include <FormulaTree.h>
```

Inheritance diagram for LogNode:



Public Member Functions

- `float Calculate (float x) const` override
 Calculates the value of the logarithm with any base at a given point.
- `std::unique_ptr< Node > Clone ()` const override
 Creates a deep copy of the node.
- `std::unique_ptr< Node > GetDerivative ()` const override
 Generates the derivative tree for this node.
- `std::string ToString ()` const override
 Converts the tree structure into a string.
- `BinaryNode (std::unique_ptr< Node > left_node=nullptr, std::unique_ptr< Node > right_node=nullptr)`

Public Member Functions inherited from [BinaryNode](#)

- [BinaryNode](#) (std::unique_ptr< [Node](#) > left_node=nullptr, std::unique_ptr< [Node](#) > right_node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [BinaryNode](#)

- std::unique_ptr< [Node](#) > left_child
- std::unique_ptr< [Node](#) > right_child

6.16.1 Detailed Description

[Node](#) representing a logarithm operation with any base.

6.16.2 Member Function Documentation

6.16.2.1 BinaryNode()

```
BinaryNode::BinaryNode (
    std::unique_ptr< Node > left_node = nullptr,
    std::unique_ptr< Node > right_node = nullptr) [inline]
```

6.16.2.2 Calculate()

```
float LogNode::Calculate (
    float x) const [inline], [override], [virtual]
```

Calculates the value of the logarithm with any base at a given point.

- This uses a logarithmic formula: $\log(a, b) = \ln(a) / \ln(b)$.

Returns

- float The calculated result of the function.

Implements [BinaryNode](#).

6.16.2.3 Clone()

`std::unique_ptr< Node > LogNode::Clone () const` `[inline]`, `[override]`, `[virtual]`

Creates a deep copy of the node.

Returns

- `std::unique_ptr<Node>` A smart pointer to the cloned node.

Implements [BinaryNode](#).

6.16.2.4 GetDerivative()

`std::unique_ptr< Node > LogNode::GetDerivative () const` `[inline]`, `[override]`, `[virtual]`

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- `std::unique_ptr<Node>` A smart pointer to the root of the derivative tree.

Implements [BinaryNode](#).

6.16.2.5 ToString()

`std::string LogNode::ToString () const` `[inline]`, `[override]`, `[virtual]`

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [BinaryNode](#).

The documentation for this class was generated from the following file:

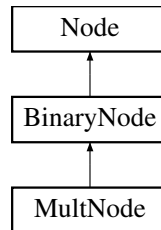
- [FormulaTree.h](#)

6.17 MultNode Class Reference

[Node](#) representing a multiplication operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for MultNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node. Uses the formula $(f * g)' = f' * g + f * g'$.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [BinaryNode](#) (std::unique_ptr< [Node](#) > left_node=nullptr, std::unique_ptr< [Node](#) > right_node=nullptr)

Public Member Functions inherited from [BinaryNode](#)

- [BinaryNode](#) (std::unique_ptr< [Node](#) > left_node=nullptr, std::unique_ptr< [Node](#) > right_node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [BinaryNode](#)

- std::unique_ptr< [Node](#) > [left_child](#)
- std::unique_ptr< [Node](#) > [right_child](#)

6.17.1 Detailed Description

[Node](#) representing a multiplication operation.

6.17.2 Member Function Documentation

6.17.2.1 BinaryNode()

```
BinaryNode::BinaryNode (
    std::unique_ptr< Node > left_node = nullptr,
    std::unique_ptr< Node > right_node = nullptr) [inline]
```

6.17.2.2 Calculate()

```
float MultNode::Calculate (
    float x) const [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [BinaryNode](#).

6.17.2.3 Clone()

```
std::unique_ptr< Node > MultNode::Clone () const [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [BinaryNode](#).

6.17.2.4 GetDerivative()

```
std::unique_ptr< Node > MultNode::GetDerivative () const [inline], [override], [virtual]
```

Generates the derivative tree for this node. Uses the formula $(f * g)' = f' * g + f * g'$.

Returns

- std::unique_ptr<Node> A smart pointer to an addition node.

Implements [BinaryNode](#).

6.17.2.5 ToString()

```
std::string MultNode::ToString () const [inline], [override], [virtual]
```

Converts the tree structure into a string.

Returns

std::string The string representation of the formula.

Implements [BinaryNode](#).

The documentation for this class was generated from the following file:

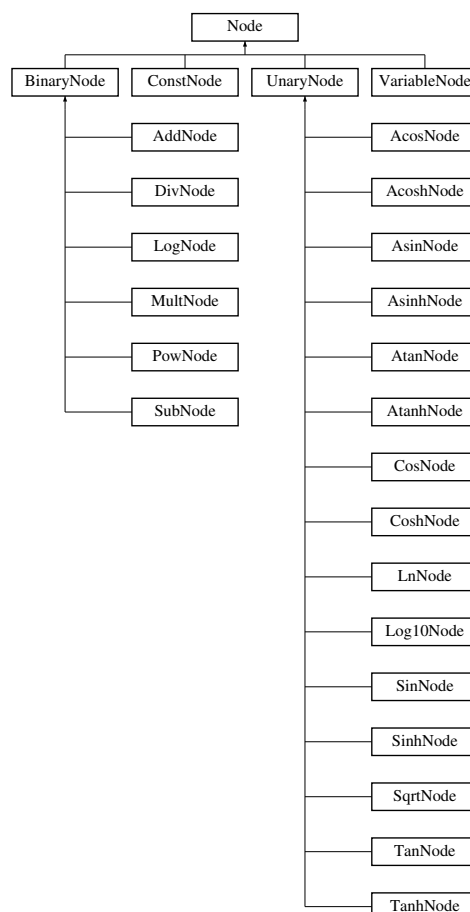
- [FormulaTree.h](#)

6.18 Node Class Reference

Base node class for the mathematical formula tree.

```
#include <FormulaTree.h>
```

Inheritance diagram for Node:



Public Member Functions

- virtual float [Calculate](#) (float x) const =0
Calculates the value of the function at a given point.
- virtual std::unique_ptr< [Node](#) > [Clone](#) () const =0
Creates a deep copy of the node.
- virtual std::unique_ptr< [Node](#) > [GetDerivative](#) () const =0
Generates the derivative tree for this node.
- virtual std::string [ToString](#) () const =0
Converts the tree structure into a string.
- virtual [~Node](#) ()=default

6.18.1 Detailed Description

Base node class for the mathematical formula tree.

- This class defines the interface required for evaluating the formula, duplicating the tree structure, and generating a derivative tree.

6.18.2 Constructor & Destructor Documentation

6.18.2.1 ~Node()

virtual Node::~~Node () [virtual], [default]

6.18.3 Member Function Documentation

6.18.3.1 Calculate()

virtual float Node::Calculate (
float x) const [pure virtual]

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implemented in [AcoshNode](#), [AcosNode](#), [AddNode](#), [AsinhNode](#), [AsinNode](#), [AtanhNode](#), [AtanNode](#), [BinaryNode](#), [ConstNode](#), [CoshNode](#), [CosNode](#), [DivNode](#), [LnNode](#), [Log10Node](#), [LogNode](#), [MultNode](#), [PowNode](#), [SinhNode](#), [SinNode](#), [SqrtNode](#), [SubNode](#), [TanhNode](#), [TanNode](#), [UnaryNode](#), and [VariableNode](#).

6.18.3.2 Clone()

virtual std::unique_ptr< [Node](#) > Node::Clone () const [pure virtual]

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implemented in [AcoshNode](#), [AcosNode](#), [AddNode](#), [AsinhNode](#), [AsinNode](#), [AtanhNode](#), [AtanNode](#), [BinaryNode](#), [ConstNode](#), [CoshNode](#), [CosNode](#), [DivNode](#), [LnNode](#), [Log10Node](#), [LogNode](#), [MultNode](#), [PowNode](#), [SinhNode](#), [SinNode](#), [SqrtNode](#), [SubNode](#), [TanhNode](#), [TanNode](#), [UnaryNode](#), and [VariableNode](#).

6.18.3.3 GetDerivative()

virtual std::unique_ptr< [Node](#) > Node::GetDerivative () const [pure virtual]

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- std::unique_ptr<Node> A smart pointer to the root of the derivative tree.

Implemented in [AcoshNode](#), [AcosNode](#), [AddNode](#), [AsinhNode](#), [AsinNode](#), [AtanhNode](#), [AtanNode](#), [BinaryNode](#), [ConstNode](#), [CoshNode](#), [CosNode](#), [DivNode](#), [LnNode](#), [Log10Node](#), [LogNode](#), [MultNode](#), [PowNode](#), [SinhNode](#), [SinNode](#), [SqrtNode](#), [SubNode](#), [TanhNode](#), [TanNode](#), [UnaryNode](#), and [VariableNode](#).

6.18.3.4 ToString()

virtual std::string Node::ToString () const [pure virtual]

Converts the tree structure into a string.

Returns

std::string The string representation of the formula.

Implemented in [AcoshNode](#), [AcosNode](#), [AddNode](#), [AsinhNode](#), [AsinNode](#), [AtanhNode](#), [AtanNode](#), [BinaryNode](#), [ConstNode](#), [CoshNode](#), [CosNode](#), [DivNode](#), [LnNode](#), [Log10Node](#), [LogNode](#), [MultNode](#), [PowNode](#), [SinhNode](#), [SinNode](#), [SqrtNode](#), [SubNode](#), [TanhNode](#), [TanNode](#), [UnaryNode](#), and [VariableNode](#).

The documentation for this class was generated from the following file:

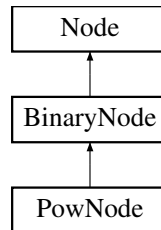
- [FormulaTree.h](#)

6.19 PowNode Class Reference

[Node](#) representing a exponentiation operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for PowNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for the power operation.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [BinaryNode](#) (std::unique_ptr< [Node](#) > left_node=nullptr, std::unique_ptr< [Node](#) > right_node=nullptr)

Public Member Functions inherited from [BinaryNode](#)

- [BinaryNode](#) (std::unique_ptr< [Node](#) > left_node=nullptr, std::unique_ptr< [Node](#) > right_node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [BinaryNode](#)

- std::unique_ptr< [Node](#) > [left_child](#)
- std::unique_ptr< [Node](#) > [right_child](#)

6.19.1 Detailed Description

[Node](#) representing a exponentiation operation.

6.19.2 Member Function Documentation

6.19.2.1 BinaryNode()

```
BinaryNode::BinaryNode (
    std::unique_ptr< Node > left_node = nullptr,
    std::unique_ptr< Node > right_node = nullptr) [inline]
```

6.19.2.2 Calculate()

```
float PowNode::Calculate (
    float x) const [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [BinaryNode](#).

6.19.2.3 Clone()

```
std::unique_ptr< Node > PowNode::Clone () const [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [BinaryNode](#).

6.19.2.4 GetDerivative()

```
std::unique_ptr< Node > PowNode::GetDerivative () const [inline], [override], [virtual]
```

Generates the derivative tree for the power operation.

- This method checks if the right child is a constant. If so, it applies the standard power rule $b * a^{(b-1)} * a'$. Otherwise, it uses the generalized logarithmic differentiation formula $a^b * ((b / a) * a' + b' * \ln(a))$.

Returns

- std::unique_ptr<Node> A smart pointer to the root of the derivative tree.

Implements [BinaryNode](#).

6.19.2.5 ToString()

```
std::string PowNode::ToString () const [inline], [override], [virtual]
```

Converts the tree structure into a string.

Returns

std::string The string representation of the formula.

Implements [BinaryNode](#).

The documentation for this class was generated from the following file:

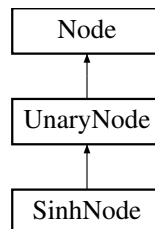
- [FormulaTree.h](#)

6.20 SinhNode Class Reference

[Node](#) representing a hyperbolic sine operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for SinhNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [UnaryNode](#)

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- std::unique_ptr< [Node](#) > child

6.20.1 Detailed Description

[Node](#) representing a hyperbolic sine operation.

6.20.2 Member Function Documentation

6.20.2.1 Calculate()

```
float SinhNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.20.2.2 Clone()

```
std::unique_ptr< Node > SinhNode::Clone () const    [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.20.2.3 GetDerivative()

```
std::unique_ptr< Node > SinhNode::GetDerivative () const [inline], [override], [virtual]
```

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- `std::unique_ptr<Node>` A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.20.2.4 ToString()

```
std::string SinhNode::ToString () const [inline], [override], [virtual]
```

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [UnaryNode](#).

6.20.2.5 UnaryNode()

```
UnaryNode::UnaryNode (  
    std::unique_ptr< Node > node = nullptr) [inline]
```

The documentation for this class was generated from the following file:

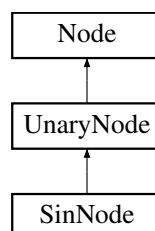
- [FormulaTree.h](#)

6.21 SinNode Class Reference

[Node](#) representing a sine operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for SinNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [UnaryNode](#)

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- std::unique_ptr< [Node](#) > [child](#)

6.21.1 Detailed Description

[Node](#) representing a sine operation.

6.21.2 Member Function Documentation

6.21.2.1 Calculate()

```
float SinNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.21.2.2 Clone()

`std::unique_ptr< Node > SinNode::Clone () const` [inline], [override], [virtual]

Creates a deep copy of the node.

Returns

- `std::unique_ptr<Node>` A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.21.2.3 GetDerivative()

`std::unique_ptr< Node > SinNode::GetDerivative () const` [inline], [override], [virtual]

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- `std::unique_ptr<Node>` A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.21.2.4 ToString()

`std::string SinNode::ToString () const` [inline], [override], [virtual]

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [UnaryNode](#).

6.21.2.5 UnaryNode()

[UnaryNode::UnaryNode](#) (
 `std::unique_ptr< Node > node = nullptr`) [inline]

The documentation for this class was generated from the following file:

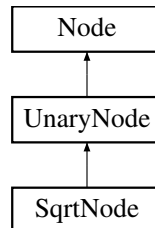
- [FormulaTree.h](#)

6.22 SqrtNode Class Reference

[Node](#) representing a square root operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for SqrtNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [UnaryNode](#)

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- std::unique_ptr< [Node](#) > [child](#)

6.22.1 Detailed Description

[Node](#) representing a square root operation.

6.22.2 Member Function Documentation

6.22.2.1 Calculate()

```
float SqrtNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.22.2.2 Clone()

```
std::unique_ptr< Node > SqrtNode::Clone () const    [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.22.2.3 GetDerivative()

```
std::unique_ptr< Node > SqrtNode::GetDerivative () const    [inline], [override], [virtual]
```

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- std::unique_ptr<Node> A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.22.2.4 ToString()

```
std::string SqrtNode::ToString () const [inline], [override], [virtual]
```

Converts the tree structure into a string.

Returns

std::string The string representation of the formula.

Implements [UnaryNode](#).

6.22.2.5 UnaryNode()

```
UnaryNode::UnaryNode (
    std::unique_ptr< Node > node = nullptr) [inline]
```

The documentation for this class was generated from the following file:

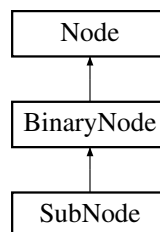
- [FormulaTree.h](#)

6.23 SubNode Class Reference

[Node](#) representing a subtraction operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for SubNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [BinaryNode](#) (std::unique_ptr< [Node](#) > left_node=nullptr, std::unique_ptr< [Node](#) > right_node=nullptr)

Public Member Functions inherited from [BinaryNode](#)

- [BinaryNode](#) (std::unique_ptr< [Node](#) > left_node=nullptr, std::unique_ptr< [Node](#) > right_node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [BinaryNode](#)

- std::unique_ptr< [Node](#) > left_child
- std::unique_ptr< [Node](#) > right_child

6.23.1 Detailed Description

[Node](#) representing a subtraction operation.

6.23.2 Member Function Documentation

6.23.2.1 BinaryNode()

```
BinaryNode::BinaryNode (
    std::unique_ptr< Node > left_node = nullptr,
    std::unique_ptr< Node > right_node = nullptr)  [inline]
```

6.23.2.2 Calculate()

```
float SubNode::Calculate (
    float x) const  [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [BinaryNode](#).

6.23.2.3 Clone()

`std::unique_ptr< Node > SubNode::Clone () const` `[inline]`, `[override]`, `[virtual]`

Creates a deep copy of the node.

Returns

- `std::unique_ptr<Node>` A smart pointer to the cloned node.

Implements [BinaryNode](#).

6.23.2.4 GetDerivative()

`std::unique_ptr< Node > SubNode::GetDerivative () const` `[inline]`, `[override]`, `[virtual]`

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- `std::unique_ptr<Node>` A smart pointer to the root of the derivative tree.

Implements [BinaryNode](#).

6.23.2.5 ToString()

`std::string SubNode::ToString () const` `[inline]`, `[override]`, `[virtual]`

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [BinaryNode](#).

The documentation for this class was generated from the following file:

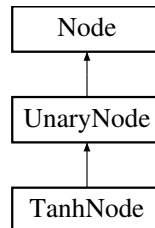
- [FormulaTree.h](#)

6.24 TanhNode Class Reference

[Node](#) representing a hyperbolic tangent operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for TanhNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.
- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [UnaryNode](#)

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- std::unique_ptr< [Node](#) > [child](#)

6.24.1 Detailed Description

[Node](#) representing a hyperbolic tangent operation.

6.24.2 Member Function Documentation

6.24.2.1 Calculate()

```
float TanhNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.24.2.2 Clone()

```
std::unique_ptr< Node > TanhNode::Clone () const    [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.24.2.3 GetDerivative()

```
std::unique_ptr< Node > TanhNode::GetDerivative () const    [inline], [override], [virtual]
```

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- std::unique_ptr<Node> A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.24.2.4 ToString()

`std::string TanhNode::ToString () const [inline], [override], [virtual]`

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [UnaryNode](#).

6.24.2.5 UnaryNode()

`UnaryNode::UnaryNode (`
`std::unique_ptr< Node > node = nullptr) [inline]`

The documentation for this class was generated from the following file:

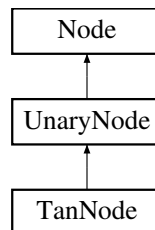
- [FormulaTree.h](#)

6.25 TanNode Class Reference

[Node](#) representing a tangent operation.

```
#include <FormulaTree.h>
```

Inheritance diagram for TanNode:



Public Member Functions

- `float Calculate (float x) const override`
 Calculates the value of the function at a given point.
- `std::unique_ptr< Node > Clone () const override`
 Creates a deep copy of the node.
- `std::unique_ptr< Node > GetDerivative () const override`
 Generates the derivative tree for this node.
- `std::string ToString () const override`
 Converts the tree structure into a string.
- `UnaryNode (std::unique_ptr< Node > node=nullptr)`

Public Member Functions inherited from [UnaryNode](#)

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Additional Inherited Members

Protected Attributes inherited from [UnaryNode](#)

- std::unique_ptr< [Node](#) > [child](#)

6.25.1 Detailed Description

[Node](#) representing a tangent operation.

6.25.2 Member Function Documentation

6.25.2.1 Calculate()

```
float TanNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [UnaryNode](#).

6.25.2.2 Clone()

```
std::unique_ptr< Node > TanNode::Clone () const    [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [UnaryNode](#).

6.25.2.3 GetDerivative()

```
std::unique_ptr< Node > TanNode::GetDerivative () const [inline], [override], [virtual]
```

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- `std::unique_ptr<Node>` A smart pointer to the root of the derivative tree.

Implements [UnaryNode](#).

6.25.2.4 ToString()

```
std::string TanNode::ToString () const [inline], [override], [virtual]
```

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [UnaryNode](#).

6.25.2.5 UnaryNode()

```
UnaryNode::UnaryNode (  
    std::unique_ptr< Node > node = nullptr) [inline]
```

The documentation for this class was generated from the following file:

- [FormulaTree.h](#)

6.26 Token Struct Reference

Represents a single token generated during the parsing of a formula.

```
#include <Parser.h>
```

Public Types

- enum class [TokenType](#) {
[VARIABLE](#) , [CONSTANT](#) , [OPERATOR](#) , [UNARY_FUN](#) ,
[BINARY_FUN](#) , [PARENTHESIS_L](#) , [PARENTHESIS_R](#) , [COMMA](#) }
 Defines the specific types of tokens recognized by the parser.

Public Member Functions

- `Token` (`Token::TokenType t_type=TokenType::CONSTANT`, `const std::string &str=""`)
Default and parameterized constructor for the `Token`.
- `Token` (`const Token &t`)
Copy constructor.
- `Token & operator=` (`const Token &t`)
Copy assignment operator.

Public Attributes

- `TokenType token_type`
The type of this token.
- `std::string val`
The representation of the token.

6.26.1 Detailed Description

Represents a single token generated during the parsing of a formula.

- The tokenizer breaks down the user input into a sequence of tokens, which are later processed to build the Abstract Syntax Tree (AST).

6.26.2 Member Enumeration Documentation

6.26.2.1 TokenType

```
enum class Token::TokenType [strong]
```

Defines the specific types of tokens recognized by the parser.

Enumerator

VARIABLE	Represents the variable 'x'.
CONSTANT	Represents a constant number.
OPERATOR	Represents a basic arithmetic operator (+, -, *, /).
UNARY_FUN	Represents a function taking a single argument.
BINARY_FUN	Represents a function taking two arguments.
PARENTHESIS_L	Represents a left opening parenthesis '('.
PARENTHESIS_R	Represents a right closing parenthesis ')'
COMMA	Represents a comma separator for multi-argument functions.

6.26.3 Constructor & Destructor Documentation

6.26.3.1 Token() [1/2]

```
Token::Token (
    TokenType t_type = TokenType::CONSTANT,
    const std::string & str = "") [inline]
```

Default and parameterized constructor for the [Token](#).

- Parameters

t_type	The type of the token. Defaults to TokenType::CONSTANT .
str	The string value of the token. Defaults to an empty string.

6.26.3.2 Token() [2/2]

```
Token::Token (
    const Token & t) [inline]
```

Copy constructor.

- Parameters

t	Existing Token to copy from.
---	--

6.26.4 Member Function Documentation

6.26.4.1 operator=()

```
Token & Token::operator= (
    const Token & t) [inline]
```

Copy assignment operator.

- Parameters

t	Token to assign from.
---	---------------------------------------

Returns

[Token&](#) A reference to this [Token](#).

6.26.5 Member Data Documentation

6.26.5.1 token_type

[TokenType](#) Token::token_type

The type of this token.

6.26.5.2 val

std::string Token::val

The representation of the token.

The documentation for this struct was generated from the following file:

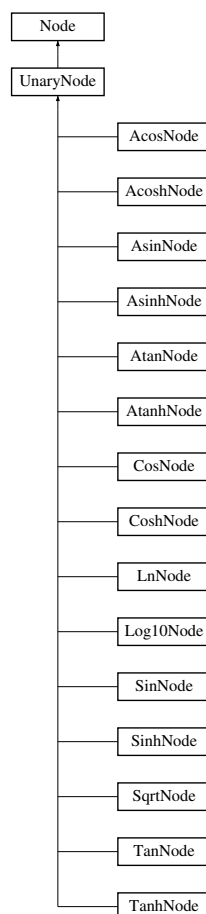
- [Parser.h](#)

6.27 UnaryNode Class Reference

Abstract base class representing unary mathematical operations.

```
#include <FormulaTree.h>
```

Inheritance diagram for UnaryNode:



Public Member Functions

- [UnaryNode](#) (std::unique_ptr< [Node](#) > node=nullptr)
- virtual float [Calculate](#) (float x) const =0
Calculates the value of the function at a given point.
- virtual std::unique_ptr< [Node](#) > [Clone](#) () const =0
Creates a deep copy of the node.
- virtual std::unique_ptr< [Node](#) > [GetDerivative](#) () const =0
Generates the derivative tree for this node.
- virtual std::string [ToString](#) () const =0
Converts the tree structure into a string.

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

Protected Attributes

- std::unique_ptr< [Node](#) > [child](#)

6.27.1 Detailed Description

Abstract base class representing unary mathematical operations.

6.27.2 Constructor & Destructor Documentation

6.27.2.1 UnaryNode()

```
UnaryNode::UnaryNode (
    std::unique_ptr< Node > node = nullptr) [inline]
```

6.27.3 Member Function Documentation

6.27.3.1 Calculate()

```
virtual float UnaryNode::Calculate (
    float x) const [pure virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [Node](#).

Implemented in [AcoshNode](#), [AcosNode](#), [AsinhNode](#), [AsinNode](#), [AtanhNode](#), [AtanNode](#), [CoshNode](#), [CosNode](#), [LnNode](#), [Log10Node](#), [SinhNode](#), [SinNode](#), [SqrtNode](#), [TanhNode](#), and [TanNode](#).

6.27.3.2 Clone()

```
virtual std::unique_ptr< Node > UnaryNode::Clone () const [pure virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [Node](#).

Implemented in [AcoshNode](#), [AcosNode](#), [AsinhNode](#), [AsinNode](#), [AtanhNode](#), [AtanNode](#), [CoshNode](#), [CosNode](#), [LnNode](#), [Log10Node](#), [SinhNode](#), [SinNode](#), [SqrtNode](#), [TanhNode](#), and [TanNode](#).

6.27.3.3 GetDerivative()

```
virtual std::unique_ptr< Node > UnaryNode::GetDerivative () const [pure virtual]
```

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- std::unique_ptr<Node> A smart pointer to the root of the derivative tree.

Implements [Node](#).

Implemented in [AcoshNode](#), [AcosNode](#), [AsinhNode](#), [AsinNode](#), [AtanhNode](#), [AtanNode](#), [CoshNode](#), [CosNode](#), [LnNode](#), [Log10Node](#), [SinhNode](#), [SinNode](#), [SqrtNode](#), [TanhNode](#), and [TanNode](#).

6.27.3.4 ToString()

virtual std::string UnaryNode::ToString () const [pure virtual]

Converts the tree structure into a string.

Returns

std::string The string representation of the formula.

Implements [Node](#).

Implemented in [AcoshNode](#), [AcosNode](#), [AsinhNode](#), [AsinNode](#), [AtanhNode](#), [AtanNode](#), [CoshNode](#), [CosNode](#), [LnNode](#), [Log10Node](#), [SinhNode](#), [SinNode](#), [SqrtNode](#), [TanhNode](#), and [TanNode](#).

6.27.4 Member Data Documentation

6.27.4.1 child

std::unique_ptr<[Node](#)> UnaryNode::child [protected]

The documentation for this class was generated from the following file:

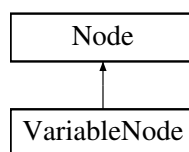
- [FormulaTree.h](#)

6.28 VariableNode Class Reference

[Node](#) of the formula tree for variable x.

```
#include <FormulaTree.h>
```

Inheritance diagram for VariableNode:



Public Member Functions

- float [Calculate](#) (float x) const override
Calculates the value of the function at a given point.
- std::unique_ptr< [Node](#) > [Clone](#) () const override
Creates a deep copy of the node.
- std::unique_ptr< [Node](#) > [GetDerivative](#) () const override
Generates the derivative tree for this node.
- std::string [ToString](#) () const override
Converts the tree structure into a string.

Public Member Functions inherited from [Node](#)

- virtual [~Node](#) ()=default

6.28.1 Detailed Description

[Node](#) of the formula tree for variable x.

6.28.2 Member Function Documentation

6.28.2.1 Calculate()

```
float VariableNode::Calculate (
    float x) const    [inline], [override], [virtual]
```

Calculates the value of the function at a given point.

- First, it recursively calls the Calculate method of its child nodes (if they exist), and then performs its own mathematical operation using those returned values.

- Parameters

x	The input value for the variable x.
---	-------------------------------------

Returns

float The calculated result of the function.

Implements [Node](#).

6.28.2.2 Clone()

```
std::unique_ptr< Node > VariableNode::Clone () const    [inline], [override], [virtual]
```

Creates a deep copy of the node.

Returns

- std::unique_ptr<Node> A smart pointer to the cloned node.

Implements [Node](#).

6.28.2.3 GetDerivative()

`std::unique_ptr< Node > VariableNode::GetDerivative () const` `[inline]`, `[override]`, `[virtual]`

Generates the derivative tree for this node.

- Applies derivation rules to construct a new syntax tree representing the mathematical derivative of the function.

Returns

- `std::unique_ptr<Node>` A smart pointer to the root of the derivative tree.

Implements [Node](#).

6.28.2.4 ToString()

`std::string VariableNode::ToString () const` `[inline]`, `[override]`, `[virtual]`

Converts the tree structure into a string.

Returns

`std::string` The string representation of the formula.

Implements [Node](#).

The documentation for this class was generated from the following file:

- [FormulaTree.h](#)

Chapter 7

File Documentation

7.1 CalculatorApp.cpp File Reference

```
#include "CalculatorApp.h"
#include <cmath>
#include "Parser.h"
```

7.2 CalculatorApp.h File Reference

```
#include <vector>
#include "External/ImGui/imgui.h"
#include "External/ImGui/implot.h"
#include "External/ImGui/imgui_stdlib.h"
#include "FormulaTree.h"
```

Classes

- class [CalculatorApp](#)
Class responsible for rendering the elements of an app and drawing a plot.

7.3 CalculatorApp.h

[Go to the documentation of this file.](#)

```
00001 #ifndef CalculatorApp_h
00002 #define CalculatorApp_h
00003
00004 #include <vector>
00005 #include "External/ImGui/imgui.h"
00006 #include "External/ImGui/implot.h"
00007 #include "External/ImGui/imgui_stdlib.h"
00008 #include "FormulaTree.h"
00009
00016 class CalculatorApp
00017 {
00018     //Constants
00019     static constexpr int STEPS = 10001;
```

```

00020     static constexpr int BUFFER_SIZE = 64;
00021
00022     //Input field flags and parameters
00023     bool input_focus = true;
00024     bool formula_changed = false;
00025     int last_cursor_position = 0;
00026     bool show_help_window = false;
00027     bool parsing_error = false;
00028
00029     //Plot data
00030     bool pos_reset = false;
00031     std::vector<float> x_points;
00032     std::vector<float> y_points;
00033     std::vector<float> x_der_points;
00034     std::vector<float> y_der_points;
00035
00036     //Text buffers
00037     std::string formula = "";
00038     std::string parsed_formula = "";
00039     std::string derivative_formula = "";
00040
00041     //Abstract Syntax Trees
00042     std::unique_ptr<Node> head_formula = nullptr;
00043     std::unique_ptr<Node> head_derivative = nullptr;
00044
00045     //GUI and Rendering State
00046     ImGuiWindowFlags window_flags;
00047     ImPlotRect prev_limits;
00048
00055     void AddSign(const char* sign);
00056
00064     bool NumberButton(const char* sign);
00065
00073     bool SignAndOperationButton(const char* sign);
00074
00082     bool FunctionButton(const char* sign);
00083
00087     void SetOfCalcButtons();
00088
00093     void TabOfFunctions(float width);
00094
00102     static int FormulaInputCallback(ImGuiInputTextCallbackData* data);
00103
00108     void RenderHelpMenu();
00109
00116     void ParseFormula();
00117
00124     void ShowFormulaPlot();
00125
00126 public:
00127
00133     CalculatorApp();
00134
00138     void Render();
00139 };
00140
00141 #endif // !CalculatorApp_h

```

7.4 CHANGELOG.md File Reference

7.5 FormulaTree.h File Reference

```

#include <cmath>
#include <sstream>
#include <memory>

```

Classes

- class [Node](#)
Base node class for the mathematical formula tree.
- class [ConstNode](#)

- [Node](#) of the formula tree for constant value.
- class [VariableNode](#)
 - [Node](#) of the formula tree for variable x.
- class [BinaryNode](#)
 - Abstract base class representing binary mathematical operations.
- class [AddNode](#)
 - [Node](#) representing an addition operation.
- class [SubNode](#)
 - [Node](#) representing a subtraction operation.
- class [MultNode](#)
 - [Node](#) representing a multiplication operation.
- class [DivNode](#)
 - [Node](#) representing a division operation.
- class [UnaryNode](#)
 - Abstract base class representing unary mathematical operations.
- class [SqrtNode](#)
 - [Node](#) representing a square root operation.
- class [LnNode](#)
 - [Node](#) representing a natural logarithm operation.
- class [SinNode](#)
 - [Node](#) representing a sine operation.
- class [CosNode](#)
 - [Node](#) representing a cosine operation.
- class [TanNode](#)
 - [Node](#) representing a tangent operation.
- class [AsinNode](#)
 - [Node](#) representing an arcsine operation.
- class [AcosNode](#)
 - [Node](#) representing an arccosine operation.
- class [AtanNode](#)
 - [Node](#) representing an arctangent operation.
- class [SinhNode](#)
 - [Node](#) representing a hyperbolic sine operation.
- class [CoshNode](#)
 - [Node](#) representing a hyperbolic cosine operation.
- class [TanhNode](#)
 - [Node](#) representing a hyperbolic tangent operation.
- class [AsinhNode](#)
 - [Node](#) representing an inverse hyperbolic sine operation.
- class [AcoshNode](#)
 - [Node](#) representing an inverse hyperbolic cosine operation.
- class [AtanhNode](#)
 - [Node](#) representing an inverse hyperbolic tangent operation.
- class [Log10Node](#)
 - [Node](#) representing a logarithm with a base 10 operation.
- class [PowNode](#)
 - [Node](#) representing an exponentiation operation.
- class [LogNode](#)
 - [Node](#) representing a logarithm operation with any base.

7.6 FormulaTree.h

[Go to the documentation of this file.](#)

```

00001 #ifndef FormulaTree_h
00002 #define FormulaTree_h
00003
00004 #include <cmath>
00005 #include <sstream>
00006 #include <memory>
00007
00013 class Node
00014 {
00015 public:
00023     virtual float Calculate(float x) const = 0;
00024
00029     virtual std::unique_ptr<Node> Clone() const = 0;
00030
00037     virtual std::unique_ptr<Node> GetDerivative() const = 0;
00038
00043     virtual std::string ToString() const = 0;
00044
00045     virtual ~Node() = default;
00046 };
00047
00051 class ConstNode :public Node
00052 {
00053     float constant;
00054 public:
00055     ConstNode(float val) : constant(val) {};
00056     ConstNode(std::string str) {
00057         if (str == "PI") {
00058             constant = 3.14159265f;
00059         }
00060         else if (str == "e") {
00061             constant = 2.718281828f;
00062         }
00063         else {
00064             std::istringstream input(str);
00065             input >> constant;
00066         }
00067     }
00068     float Calculate(float x) const override { return constant; }
00069     std::unique_ptr<Node> Clone() const override { return std::make_unique<ConstNode>(this->constant); }
00070     std::unique_ptr<Node> GetDerivative() const override { return std::make_unique<ConstNode>(0.0f); }
00071     std::string ToString() const override {
00072         if (constant == 3.14159265f)
00073             return "PI";
00074         else if (constant == 2.718281828f)
00075             return "e";
00076         else
00077         {
00078             std::ostringstream oss;
00079             oss << constant;
00080             return oss.str();
00081         }
00082     }
00083 };
00084
00088 class VariableNode :public Node
00089 {
00090 public:
00091     float Calculate(float x) const override { return x; }
00092     std::unique_ptr<Node> Clone() const override { return std::make_unique<VariableNode>(); }
00093     std::unique_ptr<Node> GetDerivative() const override { return std::make_unique<ConstNode>(1.0f); }
00094     std::string ToString() const override { return "x"; }
00095 };
00096
00100 class BinaryNode :public Node
00101 {
00102 protected:
00103     std::unique_ptr<Node> left_child;
00104     std::unique_ptr<Node> right_child;
00105 public:
00106     BinaryNode(std::unique_ptr<Node> left_node = nullptr, std::unique_ptr<Node> right_node = nullptr)
00107         :left_child(std::move(left_node)), right_child(std::move(right_node)) {};
00107     virtual float Calculate(float x) const = 0;
00108     virtual std::unique_ptr<Node> Clone() const = 0;
00109     virtual std::unique_ptr<Node> GetDerivative() const = 0;
00110     virtual std::string ToString() const = 0;
00111 };
00112
00116 class AddNode :public BinaryNode
00117 {
00118 public:
00119     using BinaryNode::BinaryNode;

```

```

00120 float Calculate(float x) const override { return left_child->Calculate(x) + right_child->Calculate(x); }
00121 std::unique_ptr<Node> Clone() const override { return std::make_unique<AddNode>(left_child->Clone(),
right_child->Clone()); }
00122 std::unique_ptr<Node> GetDerivative() const override { return
std::make_unique<AddNode>(left_child->GetDerivative(), right_child->GetDerivative()); }
00123 std::string ToString() const override {
00124     if (dynamic_cast<ConstNode*>(right_child.get()) != nullptr && right_child->Calculate(0.0f) == 0.0f)
00125         return left_child->ToString();
00126     if (dynamic_cast<ConstNode*>(left_child.get()) != nullptr && left_child->Calculate(0.0f) == 0.0f)
00127         return right_child->ToString();
00128     return "(" + left_child->ToString() + "+" + right_child->ToString() + ")";
00129 }
00130 };
00131
00135 class SubNode :public BinaryNode
00136 {
00137 public:
00138     using BinaryNode::BinaryNode;
00139     float Calculate(float x) const override { return left_child->Calculate(x) - right_child->Calculate(x); }
00140     std::unique_ptr<Node> Clone() const override { return std::make_unique<SubNode>(left_child->Clone(),
right_child->Clone()); }
00141     std::unique_ptr<Node> GetDerivative() const override { return
std::make_unique<SubNode>(left_child->GetDerivative(), right_child->GetDerivative()); }
00142     std::string ToString() const override {
00143         if (dynamic_cast<ConstNode*>(right_child.get()) != nullptr && right_child->Calculate(0.0f) == 0.0f)
00144             return left_child->ToString();
00145         if (dynamic_cast<ConstNode*>(left_child.get()) != nullptr && left_child->Calculate(0.0f) == 0.0f)
00146             return right_child->ToString();
00147         return "(" + left_child->ToString() + "-" + right_child->ToString() + ")";
00148     }
00149 };
00150
00154 class MultNode :public BinaryNode
00155 {
00156 public:
00157     using BinaryNode::BinaryNode;
00158     float Calculate(float x) const override { return left_child->Calculate(x) * right_child->Calculate(x); }
00159     std::unique_ptr<Node> Clone() const override { return std::make_unique<MultNode>(left_child->Clone(),
right_child->Clone()); }
00160
00166     std::unique_ptr<Node> GetDerivative() const override { return
std::make_unique<AddNode>(std::make_unique<MultNode>(left_child->GetDerivative(), right_child->Clone()),
std::make_unique<MultNode>(left_child->Clone(), right_child->GetDerivative())); }
00167     std::string ToString() const override {
00168         if (dynamic_cast<ConstNode*>(right_child.get()) != nullptr && right_child->Calculate(0.0f) == 1.0f)
00169             return left_child->ToString();
00170         if (dynamic_cast<ConstNode*>(left_child.get()) != nullptr && left_child->Calculate(0.0f) == 1.0f)
00171             return right_child->ToString();
00172         if (dynamic_cast<ConstNode*>(right_child.get()) != nullptr && right_child->Calculate(0.0f) == 0.0f)
00173             return "0";
00174         if (dynamic_cast<ConstNode*>(left_child.get()) != nullptr && left_child->Calculate(0.0f) == 0.0f)
00175             return "0";
00176         return "(" + left_child->ToString() + "x" + right_child->ToString() + ")";
00177     }
00178 };
00179
00183 class DivNode :public BinaryNode
00184 {
00185 public:
00186     using BinaryNode::BinaryNode;
00187
00194     float Calculate(float x) const override
00195     {
00196         float denominator = right_child->Calculate(x);
00197         if (denominator == 0.0f)
00198             return NAN;
00199         return left_child->Calculate(x) / denominator;
00200     }
00201     std::unique_ptr<Node> Clone() const override { return std::make_unique<DivNode>(left_child->Clone(),
right_child->Clone()); }
00202
00208     std::unique_ptr<Node> GetDerivative() const override { return
std::make_unique<DivNode>(std::make_unique<SubNode>(std::make_unique<MultNode>(left_child->GetDerivative(),
right_child->Clone()), std::make_unique<MultNode>(left_child->Clone(), right_child->GetDerivative())),
std::make_unique<MultNode>(right_child->Clone(), right_child->Clone())); }
00209     std::string ToString() const override {
00210         if (dynamic_cast<ConstNode*>(right_child.get()) != nullptr && right_child->Calculate(0.0f) == 1.0f)
00211             return left_child->ToString();
00212         else if (dynamic_cast<ConstNode*>(left_child.get()) != nullptr && left_child->Calculate(0.0f) == 0.0f)
00213             return right_child->ToString();
00214         return "(" + left_child->ToString() + "÷" + right_child->ToString() + ")";
00215     }
00216 };
00217
00221 class UnaryNode :public Node
00222 {
00223 protected:

```

```

00224     std::unique_ptr<Node> child;
00225 public:
00226     UnaryNode(std::unique_ptr<Node> node = nullptr) :child(std::move(node)) {};
00227     virtual float Calculate(float x) const = 0;
00228     virtual std::unique_ptr<Node> Clone() const = 0;
00229     virtual std::unique_ptr<Node> GetDerivative() const = 0;
00230     virtual std::string ToString() const = 0;
00231 };
00232
00236 class SqrtNode :public UnaryNode
00237 {
00238 public:
00239     using UnaryNode::UnaryNode;
00240     float Calculate(float x) const override { return sqrt(child->Calculate(x)); }
00241     std::unique_ptr<Node> Clone() const override { return std::make_unique<SqrtNode>(child->Clone()); }
00242     std::unique_ptr<Node> GetDerivative() const override { return std::make_unique<DivNode>(child->GetDerivative(),
std::make_unique<MultNode>(std::make_unique<ConstNode>(2.0f), this->Clone())); }
00243     std::string ToString() const override { return "sqrt(" + child->ToString() + ")"; }
00244 };
00245
00249 class LnNode :public UnaryNode
00250 {
00251 public:
00252     using UnaryNode::UnaryNode;
00253     float Calculate(float x) const override { return log(child->Calculate(x)); }
00254     std::unique_ptr<Node> Clone() const override { return std::make_unique<LnNode>(child->Clone()); }
00255     std::unique_ptr<Node> GetDerivative() const override { return std::make_unique<DivNode>(child->GetDerivative(),
child->Clone()); }
00256     std::string ToString() const override { return "ln(" + child->ToString() + ")"; }
00257 };
00258
00259 class CosNode;
00260
00264 class SinNode :public UnaryNode
00265 {
00266 public:
00267     using UnaryNode::UnaryNode;
00268     float Calculate(float x) const override { return sin(child->Calculate(x)); }
00269     std::unique_ptr<Node> Clone() const override { return std::make_unique<SinNode>(child->Clone()); }
00270     std::unique_ptr<Node> GetDerivative() const override;
00271     std::string ToString() const override { return "sin(" + child->ToString() + ")"; }
00272 };
00273
00277 class CosNode :public UnaryNode
00278 {
00279 public:
00280     using UnaryNode::UnaryNode;
00281     float Calculate(float x) const override { return cos(child->Calculate(x)); }
00282     std::unique_ptr<Node> Clone() const override { return std::make_unique<CosNode>(child->Clone()); }
00283     std::unique_ptr<Node> GetDerivative() const override { return
std::make_unique<MultNode>(std::make_unique<SubNode>(std::make_unique<ConstNode>(0.0f),
std::make_unique<SinNode>(child->Clone())), child->GetDerivative()); }
00284     std::string ToString() const override { return "cos(" + child->ToString() + ")"; }
00285 };
00286
00287 inline std::unique_ptr<Node> SinNode::GetDerivative() const { return
std::make_unique<MultNode>(std::make_unique<CosNode>(child->Clone()), child->GetDerivative()); }
00288
00292 class TanNode :public UnaryNode
00293 {
00294 public:
00295     using UnaryNode::UnaryNode;
00296     float Calculate(float x) const override { return tan(child->Calculate(x)); }
00297     std::unique_ptr<Node> Clone() const override { return std::make_unique<TanNode>(child->Clone()); }
00298     std::unique_ptr<Node> GetDerivative() const override { return std::make_unique<DivNode>(child->GetDerivative(),
std::make_unique<MultNode>(std::make_unique<CosNode>(child->Clone()),
std::make_unique<CosNode>(child->Clone()))); }
00299     std::string ToString() const override { return "tan(" + child->ToString() + ")"; }
00300 };
00301
00305 class AsinNode :public UnaryNode
00306 {
00307 public:
00308     using UnaryNode::UnaryNode;
00309     float Calculate(float x) const override { return asin(child->Calculate(x)); }
00310     std::unique_ptr<Node> Clone() const override { return std::make_unique<AsinNode>(child->Clone()); }
00311     std::unique_ptr<Node> GetDerivative() const override { return std::make_unique<DivNode>(child->GetDerivative(),
std::make_unique<SqrtNode>(std::make_unique<SubNode>(std::make_unique<ConstNode>(1.0f),
std::make_unique<MultNode>(child->Clone(), child->Clone())))); }
00312     std::string ToString() const override { return "arcsin(" + child->ToString() + ")"; }
00313 };
00314
00318 class AcosNode :public UnaryNode
00319 {
00320 public:
00321     using UnaryNode::UnaryNode;
00322     float Calculate(float x) const override { return acos(child->Calculate(x)); }

```

```

00323     std::unique_ptr<Node> Clone() const override { return std::make_unique<AcosNode>(child->Clone()); }
00324     std::unique_ptr<Node> GetDerivative() const override { return
        std::make_unique<SubNode>(std::make_unique<ConstNode>(0.0f),
        std::make_unique<DivNode>(child->GetDerivative(),
        std::make_unique<SqrtNode>(std::make_unique<SubNode>(std::make_unique<ConstNode>(1.0f),
        std::make_unique<MultNode>(child->Clone(), child->Clone())))); }
00325     std::string ToString() const override { return "arccos(" + child->ToString() + ")"; }
00326 };
00327
00331 class AtanNode :public UnaryNode
00332 {
00333 public:
00334     using UnaryNode::UnaryNode;
00335     float Calculate(float x) const override { return atan(child->Calculate(x)); }
00336     std::unique_ptr<Node> Clone() const override { return std::make_unique<AtanNode>(child->Clone()); }
00337     std::unique_ptr<Node> GetDerivative() const override { return std::make_unique<DivNode>(child->GetDerivative(),
        std::make_unique<AddNode>(std::make_unique<ConstNode>(1.0f), std::make_unique<MultNode>(child->Clone(),
        child->Clone()))); }
00338     std::string ToString() const override { return "arctan(" + child->ToString() + ")"; }
00339 };
00340
00341 class CoshNode;
00342
00346 class SinhNode :public UnaryNode
00347 {
00348 public:
00349     using UnaryNode::UnaryNode;
00350     float Calculate(float x) const override { return sinh(child->Calculate(x)); }
00351     std::unique_ptr<Node> Clone() const override { return std::make_unique<SinhNode>(child->Clone()); }
00352     std::unique_ptr<Node> GetDerivative() const override;
00353     std::string ToString() const override { return "sinh(" + child->ToString() + ")"; }
00354 };
00355
00359 class CoshNode :public UnaryNode
00360 {
00361 public:
00362     using UnaryNode::UnaryNode;
00363     float Calculate(float x) const override { return cosh(child->Calculate(x)); }
00364     std::unique_ptr<Node> Clone() const override { return std::make_unique<CoshNode>(child->Clone()); }
00365     std::unique_ptr<Node> GetDerivative() const override { return
        std::make_unique<MultNode>(std::make_unique<SinhNode>(child->Clone()), child->GetDerivative()); }
00366     std::string ToString() const override { return "cosh(" + child->ToString() + ")"; }
00367 };
00368
00369 inline std::unique_ptr<Node> SinhNode::GetDerivative() const { return
        std::make_unique<MultNode>(std::make_unique<CoshNode>(child->Clone()), child->GetDerivative()); }
00370
00374 class TanhNode :public UnaryNode
00375 {
00376 public:
00377     using UnaryNode::UnaryNode;
00378     float Calculate(float x) const override { return tanh(child->Calculate(x)); }
00379     std::unique_ptr<Node> Clone() const override { return std::make_unique<TanhNode>(child->Clone()); }
00380     std::unique_ptr<Node> GetDerivative() const override { return std::make_unique<DivNode>(child->GetDerivative(),
        std::make_unique<MultNode>(std::make_unique<CoshNode>(child->Clone()),
        std::make_unique<CoshNode>(child->Clone()))); }
00381     std::string ToString() const override { return "tanh(" + child->ToString() + ")"; }
00382 };
00383
00387 class AsinhNode :public UnaryNode
00388 {
00389 public:
00390     using UnaryNode::UnaryNode;
00391     float Calculate(float x) const override { return asinh(child->Calculate(x)); }
00392     std::unique_ptr<Node> Clone() const override { return std::make_unique<AsinhNode>(child->Clone()); }
00393     std::unique_ptr<Node> GetDerivative() const override { return std::make_unique<DivNode>(child->GetDerivative(),
        std::make_unique<SqrtNode>(std::make_unique<AddNode>(std::make_unique<MultNode>(child->Clone(),
        child->Clone()), std::make_unique<ConstNode>(1.0f)))); }
00394     std::string ToString() const override { return "arcsinh(" + child->ToString() + ")"; }
00395 };
00396
00400 class AcoshNode :public UnaryNode
00401 {
00402 public:
00403     using UnaryNode::UnaryNode;
00404     float Calculate(float x) const override { return acosh(child->Calculate(x)); }
00405     std::unique_ptr<Node> Clone() const override { return std::make_unique<AcoshNode>(child->Clone()); }
00406     std::unique_ptr<Node> GetDerivative() const override { return std::make_unique<DivNode>(child->GetDerivative(),
        std::make_unique<SqrtNode>(std::make_unique<SubNode>(std::make_unique<MultNode>(child->Clone(),
        child->Clone()), std::make_unique<ConstNode>(1.0f)))); }
00407     std::string ToString() const override { return "arcosh(" + child->ToString() + ")"; }
00408 };
00409
00413 class AtanhNode :public UnaryNode
00414 {
00415 public:
00416     using UnaryNode::UnaryNode;

```

```

00417     float Calculate(float x) const override { return atanh(child->Calculate(x)); }
00418     std::unique_ptr<Node> Clone() const override { return std::make_unique<AtanhNode>(child->Clone()); }
00419     std::unique_ptr<Node> GetDerivative() const override { return std::make_unique<DivNode>(child->GetDerivative(),
std::make_unique<SubNode>(std::make_unique<ConstNode>(1.0f), std::make_unique<MultNode>(child->Clone(),
child->Clone()))); }
00420     std::string ToString() const override { return "atanh(" + child->ToString() + ")"; }
00421 };
00422
00426 class Log10Node :public UnaryNode
00427 {
00428 public:
00429     using UnaryNode::UnaryNode;
00430     float Calculate(float x) const override { return log10(child->Calculate(x)); }
00431     std::unique_ptr<Node> Clone() const override { return std::make_unique<Log10Node>(child->Clone()); }
00432     std::unique_ptr<Node> GetDerivative() const override { return std::make_unique<DivNode>(child->GetDerivative(),
std::make_unique<MultNode>(child->Clone(), std::make_unique<LnNode>(std::make_unique<ConstNode>(10.0f)))); }
00433     std::string ToString() const override { return "log10(" + child->ToString() + ")"; }
00434 };
00435
00439 class PowNode :public BinaryNode
00440 {
00441 public:
00442     using BinaryNode::BinaryNode;
00443     float Calculate(float x) const override { return pow(left_child->Calculate(x), right_child->Calculate(x)); }
00444     std::unique_ptr<Node> Clone() const override { return std::make_unique<PowNode>(left_child->Clone(),
right_child->Clone()); }
00445
00453     std::unique_ptr<Node> GetDerivative() const override
00454     {
00455         if (dynamic_cast<ConstNode*>(right_child.get()) != nullptr)
00456             return std::make_unique<MultNode>(std::make_unique<MultNode>(right_child->Clone(),
std::make_unique<PowNode>(left_child->Clone(), std::make_unique<ConstNode>(right_child->Calculate(0.0f) -
1.0f))), left_child->GetDerivative());
00457         return std::make_unique<MultNode>(this->Clone(),
std::make_unique<AddNode>(std::make_unique<MultNode>(right_child->GetDerivative(),
std::make_unique<LnNode>(left_child->Clone())), std::make_unique<MultNode>(right_child->Clone(),
std::make_unique<DivNode>(left_child->GetDerivative(), left_child->Clone()))));
00458     }
00459     std::string ToString() const override {
00460         if (dynamic_cast<ConstNode*>(right_child.get()) != nullptr && right_child->Calculate(0.0f) == 1.0f)
00461             return left_child->ToString();
00462         else
00463             return "pow(" + left_child->ToString() + ", " + right_child->ToString() + ")";
00464     }
00465 };
00466
00470 class LogNode :public BinaryNode
00471 {
00472 public:
00473     using BinaryNode::BinaryNode;
00474
00480     float Calculate(float x) const override { return log(right_child->Calculate(x)) / log(left_child->Calculate(x)); }
00481     std::unique_ptr<Node> Clone() const override { return std::make_unique<LogNode>(left_child->Clone(),
right_child->Clone()); }
00482     std::unique_ptr<Node> GetDerivative() const override { return
std::make_unique<DivNode>(std::make_unique<LnNode>(right_child->Clone()),
std::make_unique<LnNode>(left_child->Clone()))->GetDerivative(); }
00483     std::string ToString() const override { return "log(" + left_child->ToString() + ", " + right_child->ToString() + ")"; }
00484 };
00485
00486 #endif // !FormulaTree_h

```

7.7 main.cpp File Reference

```

#include "External/ImGui/imgui_impl_glfw.h"
#include "External/ImGui/imgui_impl_opengl3.h"
#include "CalculatorApp.h"
#include <GLFW/glfw3.h>
#include <iostream>

```

Functions

- void `glfw_error_callback` (int error, const char *description)
- int `main` ()

7.7.1 Function Documentation

7.7.1.1 glfw_error_callback()

```
void glfw_error_callback (
    int error,
    const char * description)
```

7.7.1.2 main()

```
int main ()
```

7.8 Parser.cpp File Reference

```
#include "Parser.h"
#include <sstream>
#include <stack>
#include "FormulaTree.h"
#include <iostream>
```

Functions

- bool [is_numeric](#) (const std::string &str)

Checks if the string contains a constant.
- bool [OperatorSign](#) (const char &sign)

Checks if the string contains one of the signs : '-', '+', '*', '/', '(', ')', ',', '.'.
- bool [CheckToken](#) (const std::string &str, [Token](#) &token)

Assigns a token a proper TokenType based on the given string.
- [Token CreateOperatorToken](#) (const char sign)

Creates one of the signs token: '-', '+', '*', '/', '(', ')', ',', '.'.
- int [getPriority](#) (const std::string &str)

Determines order of operation.
- std::unique_ptr< [Node](#) > [ModifyToRPN](#) (std::vector< [Token](#) > &tokens)

Modifies a token vector to a Reverse Polish Notation order.
- std::unique_ptr< [Node](#) > [CreateBinFunNode](#) (const std::string &str, std::unique_ptr< [Node](#) > left, std::unique_ptr< [Node](#) > right)

Creates a tree node for a binary function or an arithmetic operator.
- std::unique_ptr< [Node](#) > [CreateUnFunNode](#) (const std::string &str, std::unique_ptr< [Node](#) > child)

Creates a tree node for an unary function.
- std::unique_ptr< [Node](#) > [CreateFormulaTree](#) (std::vector< [Token](#) > &RPN)

Creates an Abstract Syntax Tree as a formula tree.

7.8.1 Function Documentation

7.8.1.1 CheckToken()

```
bool CheckToken (
    const std::string & str,
    Token & token)
```

Assigns a token a proper TokenType based on the given string.

- Parameters

str	Part of the user input from which the TokenType is based.
token	Token to which it will be assigned a proper TokenType.

Returns

bool Returns true if the TokenType was assigned successfully.

7.8.1.2 CreateBinFunNode()

```
std::unique_ptr< Node > CreateBinFunNode (
    const std::string & str,
    std::unique_ptr< Node > left,
    std::unique_ptr< Node > right)
```

Creates a tree node for a binary function or an arythmetic operator.

- A proper node is created based on the string parameter. This node then points at left and right as its left and right child.

- Parameters

str	Function or an arythmetic operation.
Left	Left child of the node.
Right	child of the node.

Returns

std::unique_ptr<Node> Created node.

7.8.1.3 CreateFormulaTree()

```
std::unique_ptr< Node > CreateFormulaTree (
    std::vector< Token > & RPN)
```

Creates an Abstract Syntax Tree as a formula tree.

- Takes a vector of tokens of functions, constants and variables that are in Reverse Polish Notation order. Next it creates an AST out of it to create a formula tree.

- Parameters

RPN	vector of tokens in RPN order.
-----	--------------------------------

Returns

std::unique_ptr<Node> Root of the formula tree.

7.8.1.4 CreateOperatorToken()

```
Token CreateOperatorToken (
    const char sign)
```

Creates one of the signs token: '-', '+', '*', '/', '(', ')', ',', '.'.

- Based on the parameter it creates a proper Token.

-

Parameters

sign	Symbol that indicates which Token should be created.
------	--

Returns

Token Created Token based on a sign.

7.8.1.5 CreateUnFunNode()

```
std::unique_ptr< Node > CreateUnFunNode (
    const std::string & str,
    std::unique_ptr< Node > child)
```

Creates a tree node for an unary function.

- A proper node is created based on the string parameter. This node has a pointer set on child parameter.

- Parameters

str	Unary function.
child	Child of the node.

Returns

std::unique_ptr<Node> Created node.

7.8.1.6 getPriority()

```
int getPriority (
    const std::string & str)
```

Determines order of operation.

- Returns an integer indicating how hisg on the priority in the order of mathematical operations is given operation. The higher the number the higher the priority.

- Parameters

str	Operation that needs to be given a priority.
-----	--

Returns

int Priority number.

7.8.1.7 is_numeric()

```
bool is_numeric (
    const std::string & str)
```

Checks if the string contains a constant.

- Parameters

str	String that may contain a constant.
-----	-------------------------------------

Returns

bool Returns true if string contains a constant value.

7.8.1.8 ModifyToRPN()

```
std::unique_ptr< Node > ModifyToRPN (
    std::vector< Token > & tokens)
```

Modifies a token vector to a Reverse Polish Notation order.

- This function takes previously created a vector of Tokens and modifies the order so it can be read using the RPN. It deletes not needed tokens such as parenthesis or comma tokens.

- Parameters

tokens	Vector of tokens that were created based on the user text input.
--------	--

Returns

std::unique_ptr<Node> Root of formula tree.

7.8.1.9 OperatorSign()

```
bool OperatorSign (
    const char & sign)
```

Checks if the string contains one of the sings : '-', '+', '*', '/', '(', ')', ',', '.'.

- Parameters

str	String that may contain a sign.
-----	---------------------------------

Returns

bool Returns true if string contains a sign.

7.9 Parser.h File Reference

```
#include <string>
#include <vector>
#include "FormulaTree.h"
```

Classes

- struct [Token](#)
Represents a single token generated during the parsing of a formula.

Functions

- `bool is_numeric (const std::string &str)`
Checks if the string contains a constant.
- `bool OperatorSign (const char &sign)`
Checks if the string contains one of the signs : '-', '+', '*', '/', '(', ')', ',', '.'.
- `bool CheckToken (const std::string &str, Token &token)`
Assigns a token a proper TokenType based on the given string.
- `Token CreateOperatorToken (const char sign)`
Creates one of the signs token: '-', '+', '*', '/', '(', ')', ',', '.'.
- `int getPriority (const std::string &str)`
Determines order of operation.
- `std::unique_ptr< Node > ModifyToRPN (std::vector< Token > &tokens)`
Modifies a token vector to a Reverse Polish Notation order.
- `std::unique_ptr< Node > CreateBinFunNode (const std::string &str, std::unique_ptr< Node > left, std::unique_ptr< Node > right)`
Creates a tree node for a binary function or an arithmetical operator.
- `std::unique_ptr< Node > CreateUnFunNode (const std::string &str, std::unique_ptr< Node > child)`
Creates a tree node for a unary function.
- `std::unique_ptr< Node > CreateFormulaTree (std::vector< Token > &RPN)`
Creates an Abstract Syntax Tree as a formula tree.

7.9.1 Function Documentation

7.9.1.1 CheckToken()

```
bool CheckToken (
    const std::string & str,
    Token & token)
```

Assigns a token a proper TokenType based on the given string.

- Parameters

str	Part of the user input from which the TokenType is based.
token	Token to which it will be assigned a proper TokenType.

Returns

bool Returns true if the TokenType was assigned successfully.

7.9.1.2 CreateBinFunNode()

```
std::unique_ptr< Node > CreateBinFunNode (
    const std::string & str,
    std::unique_ptr< Node > left,
    std::unique_ptr< Node > right)
```

Creates a tree node for a binary function or an arithmetical operator.

- A proper node is created based on the string parameter. This node then points at left and right as its left and right child.

- Parameters

str	Function or an arithmetical operation.
Left	Left child of the node.
Right	child of the node.

Returns

std::unique_ptr<Node> Created node.

7.9.1.3 CreateFormulaTree()

```
std::unique_ptr< Node > CreateFormulaTree (
    std::vector< Token > & RPN)
```

Creates an Abstract Syntax Tree as a formula tree.

- Takes a vector of tokens of functions, constants and variables that are in Reverse Polish Notation order. Next it creates an AST out of it to create a formula tree.

- Parameters

RPN	vector of tokens in RPN order.
-----	--------------------------------

Returns

std::unique_ptr<Node> Root of the formula tree.

7.9.1.4 CreateOperatorToken()

```
Token CreateOperatorToken (
    const char sign)
```

Creates one of the signs token: '-', '+', '*', '/', '(', ')', '!', '.'.

- Based on the parameter it creates a proper [Token](#).

- Parameters

sign	Symbol that indicates which Token should be created.
------	--

Returns

[Token](#) Created [Token](#) based on a sign.

7.9.1.5 CreateUnFunNode()

```
std::unique_ptr< Node > CreateUnFunNode (
    const std::string & str,
    std::unique_ptr< Node > child)
```

Creates a tree node for an unary function.

- A proper node is created based on the string parameter. This node has a pointer set on child parameter.
- Parameters

str	Unary function.
child	Child of the node.

Returns

std::unique_ptr<Node> Created node.

7.9.1.6 getPriority()

```
int getPriority (
    const std::string & str)
```

Determines order of operation.

- Returns an integer indicating how hisg on the priority in the order of mathematical operations is given operation. The higher the number the higher the priority.
- Parameters

str	Operation that needs to be given a priority.
-----	--

Returns

int Priority number.

7.9.1.7 `is_numeric()`

```
bool is_numeric (
    const std::string & str)
```

Checks if the string contains a constant.

- Parameters

str	String that may contain a constant.
-----	-------------------------------------

Returns

bool Returns true if string contains a constant value.

7.9.1.8 `ModifyToRPN()`

```
std::unique_ptr< Node > ModifyToRPN (
    std::vector< Token > & tokens)
```

Modifies a token vector to a Reverse Polish Notation order.

- This function takes previously created a vector of Tokens and modifies the order so it can be read using the RPN. It deletes not needed tokens such as parenthesis or comma tokens.

- Parameters

tokens	Vector of tokens that were created based on the user text input.
--------	--

Returns

std::unique_ptr<Node> Root of formula tree.

7.9.1.9 `OperatorSign()`

```
bool OperatorSign (
    const char & sign)
```

Checks if the string contains one of the sings : '-', '+', '*', '/', '(', ')', ',', ' '.

- Parameters

str	String that may contain a sign.
-----	---------------------------------

Returns

bool Returns true if string contains a sign.

7.10 Parser.h

[Go to the documentation of this file.](#)

```

00001 #ifndef Parser_h
00002 #define Parser_h
00003
00004 #include <string>
00005 #include <vector>
00006 #include "FormulaTree.h"
00007
00013 struct Token
00014 {
00018     enum class TokenType
00019     {
00020         VARIABLE,
00021         CONSTANT,
00022         OPERATOR,
00023         UNARY_FUN,
00024         BINARY_FUN,
00025         PARENTHESIS_L,
00026         PARENTHESIS_R,
00027         COMMA
00028     };
00029
00030     TokenType token_type;
00031     std::string val;
00032
00038     Token(TokenType t_type = TokenType::CONSTANT, const std::string& str = "") : token_type(t_type),
    val(str) {};
00039
00044     Token(const Token& t) : token_type(t.token_type), val(t.val) {};
00045
00051     Token& operator=(const Token& t)
00052     {
00053         if (this == &t)
00054             return *this;
00055
00056         token_type = t.token_type;
00057         val = t.val;
00058         return *this;
00059     }
00060 };
00061
00067 bool is_numeric(const std::string& str);
00068
00074 bool OperatorSign(const char& sign);
00075
00082 bool CheckToken(const std::string& str, Token& token);
00083
00090 Token CreateOperatorToken(const char sign);
00091
00100 int getPriority(const std::string& str);
00101
00110 std::unique_ptr<Node> ModifyToRPN(std::vector<Token>& tokens);
00111
00121 std::unique_ptr<Node> CreateBinFunNode(const std::string& str, std::unique_ptr<Node> left, std::unique_ptr<Node>
    right);
00122
00131 std::unique_ptr<Node> CreateUnFunNode(const std::string& str, std::unique_ptr<Node> child);
00132
00140 std::unique_ptr<Node> CreateFormulaTree(std::vector<Token>& RPN);
00141
00142 #endif // !Parser_h

```

7.11 README.md File Reference